EC302

Digital System Design

Combined Module Notes

Modules 1 – 4

Semester III • ECE • 3 Credits

CGEC | B.Tech ECE (2023–27)

Disclaimer / Study Guide Notice:
These are condensed **Short Notes** focusing on core theory, key formulas, block/circuit diagrams, and standard viva Q&As for university exam revision. Exhaustive numerical practice problems and derivation edge cases are not covered. This serves as a quick-revision reference guide for students.

Biraj Sarkar

IN ASSOCIATION WITH

MAKAUT Wingman • **MAKAUT Future Minds**

Cooch Behar Government Engineering College

Exam-Ready Notes

Contents

Module 1: Number Systems and Binary Arithmetic 5

1 Number Systems and Binary Arithmetic . . . 5

1.1 Radix Conversions . . . 5

1.2 Representation of Signed Numbers . . . 5

1.3 2's Complement Arithmetic and Overflow Detection . . . 6

2 Boolean Algebra and Logic Simplification . . . 6

2.1 Fundamental Laws of Boolean Algebra . . . 6

2.2 De Morgan's Theorems and the Principle of Duality . . . 7

2.3 Canonical and Standard Forms . . . 7

3 Karnaugh Map (K-Map) Simplification . . . 7

3.1 Grouping Principles and Implicants . . . 8

3.2 Static Hazards in Combinational Logic . . . 8

4 Binary Codes and Code Converters . . . 8

4.1 Standard Binary Codes . . . 8

4.2 Binary to Gray and Gray to Binary Code Conversion . . . 9

5 Combinational Arithmetic Circuits . . . 9

5.1 Half Adder, Full Adder, and Subtractors . . . 9

5.2 Ripple Carry Adder vs. Carry Lookahead Adder (CLA) . . . 9

5.3 BCD Adder . . . 10

6 MSI Combinational Logic Devices . . . 10

6.1 Magnitude Comparators . . . 10

6.2 Multiplexers (Data Selectors) . . . 10

6.3 Encoders and Decoders . . . 11

6.4 Barrel Shifters and Arithmetic Logic Units (ALUs) . . . 11

7 Quick Revision Page . . . 12

Module 2: Latches and Flip-Flops 15

1 Latches and Flip-Flops . . . 15

1.1 Latches vs. Flip-Flops . . . 15

1.2 The S-R Latch and Forbidden State . . . 15

1.3 J-K Flip-Flop and the Race-Around Condition . . . 15

1.4 Master-Slave JK Flip-Flop Architecture . . . 16

1.5 Characteristic Equations and Excitation Tables . . . 17

2 Registers and Shift Registers . . . 17

2.1 Shift Register Configurations . . . 17

2.2 Universal Shift Register (IC 74194) . . . 17

3 Asynchronous (Ripple) and Synchronous Counters . . . 17

3.1 Asynchronous (Ripple) Counters . . . 18
3.2 Synchronous Counters . . . 18
3.3 Ring Counter and Johnson (Twisted Ring) Counter . . . 18
4 Finite State Machines (FSM) . . . 18
4.1 Mealy vs. Moore FSM Models . . . 19
4.2 State Reduction and State Assignment . . . 19
5 Design of Synchronous Circuits . . . 19
5.1 Sequence Detector / Pulse Train Generator . . . 20
5.2 Pseudo-Random Binary Sequence (PRBS) Generator . . . 20
6 Quick Revision Page . . . 21

Module 3: Digital Logic Families and Performance Metrics 24

1 Digital Logic Families and Performance Metrics . . . 24
1.1 Key Performance Metrics of Digital ICs . . . 24
1.2 TTL (Transistor-Transistor Logic) . . . 25
1.3 ECL (Emitter-Coupled Logic) . . . 26
1.4 CMOS Logic Family . . . 26
2 Semiconductor Memories . . . 27
2.1 Random Access Memory: SRAM vs. DRAM . . . 27
2.2 Read-Only Memories (ROM) and Flash Memory . . . 28
3 Programmable Logic Devices (PLDs) and FPGAs . . . 29
3.1 Classification of Simple PLDs (SPLDs) . . . 29
3.2 Complex Programmable Logic Devices (CPLD) . . . 30
3.3 Field Programmable Gate Arrays (FPGA) . . . 30
4 Digital-to-Analog Converters (DAC) . . . 31
4.1 DAC Specifications . . . 31
4.2 Binary-Weighted Resistor DAC . . . 32
4.3 R-$2R$ Ladder DAC . . . 32
5 Analog-to-Digital Converters (ADC) . . . 33
5.1 Quantization Error and ADC Dynamic Range . . . 33
5.2 Flash (Simultaneous / Parallel) ADC . . . 33
5.3 Successive Approximation Register (SAR) ADC . . . 33
5.4 Dual-Slope Integrating ADC . . . 34
6 Sample and Hold (S/H) Circuit . . . 35
6.1 Circuit Topology and Modes of Operation . . . 35
6.2 Key Dynamic Performance Parameters . . . 35
7 Quick Revision Page . . . 37

Module 4: The VLSI Design Flow and Methodologies 40

1 The VLSI Design Flow and Methodologies . . . 40

© Biraj

1.1 The Gajski-Kuhn Y-Chart . . . 40
1.2 Step-by-Step Modern Digital ASIC and FPGA Design Flow . . . 40
2 VHDL Language Fundamentals and Structural Units . . . 42
2.1 Fundamental Design Units of VHDL . . . 42
2.2 VHDL Port Modes . . . 43
3 Data Types, Data Objects, and Operators . . . 43
3.1 Data Objects in VHDL . . . 43
3.2 Standard and IEEE 1164 Data Types . . . 44
3.3 VHDL Operators . . . 44
4 VHDL Modeling Styles and Coding Methodologies . . . 45
4.1 Dataflow (Concurrent) Modeling . . . 45
4.2 Behavioral (Sequential) Modeling . . . 45
4.3 Structural Modeling . . . 46
5 VHDL Code Library for Combinational Logic Circuits . . . 46
6 VHDL Code Library for Sequential Logic Circuits . . . 49
7 Quick Revision Page . . . 53

© Biraj

Module 1

Number Systems and Binary Arithmetic

1. Number Systems and Binary Arithmetic

Digital systems process discrete electrical signals representing numerical data in positional number systems. A number N in a base (radix) r is represented as:

$$(N)_r = \sum_{i=-m}^{n-1} d_i \cdot r^i = d_{n-1}r^{n-1} + \cdots + d_1 r^1 + d_0 r^0 + d_{-1}r^{-1} + \cdots + d_{-m}r^{-m} \tag{1}$$

where $0 \leq d_i \leq (r-1)$, n is the number of integer digits, and m is the number of fractional digits.

1.1 Radix Conversions

- **Base-r to Decimal:** Expand the number into polynomials of powers of r.
- **Decimal to Base-r:** Divide the integer portion successively by r (collecting remainders from LSB to MSB); multiply the fractional portion successively by r (collecting integer carries from MSB to LSB).
- **Binary, Octal, and Hexadecimal Conversions:** Because $8 = 2^3$ and $16 = 2^4$, conversion between binary and octal requires grouping bits into groups of 3; conversion between binary and hexadecimal requires grouping bits into groups of 4.

1.2 Representation of Signed Numbers

In an n-bit binary system, unsigned numbers range from 0 to $2^n - 1$. Signed numbers dedicate the most significant bit (MSB) as the sign bit: MSB $= 0 \implies$ Positive, MSB $= 1 \implies$ Negative.

Three Signed Number Representations (n-bit)

1. **Signed Magnitude:** Magnitude is stored in the lower $(n-1)$ bits.

   $$\text{Range: } -(2^{n-1}-1) \leq N \leq +(2^{n-1}-1) \tag{2}$$

   Drawback: Two distinct representations for zero ($+0 = 0000_2$, $-0 = 1000_2$), complicating ALU hardware.

2. **1's Complement:** Negative numbers are formed by bitwise inverting all n bits of the positive magnitude.

   $$\text{Range: } -(2^{n-1}-1) \leq N \leq +(2^{n-1}-1) \tag{3}$$

© Biraj

Drawback: Dual representation for zero ($+0 = 0000_2$, $-0 = 1111_2$); requires an end-around carry addition.

3. **2's Complement (Universal Standard):** Negative numbers are formed by adding 1 to the 1's complement ($N_{2's} = 2^n - |N|$).

$$\text{Range: } -2^{n-1} \leq N \leq +(2^{n-1}-1) \tag{4}$$

Advantages: Single, unique representation for zero (0000_2); arithmetic subtraction is performed using standard binary addition without end-around carry; extra negative value can be represented.

1.3 2's Complement Arithmetic and Overflow Detection

To compute $A-B$, the digital processor computes $A+(\text{2's complement of } B)$. An end-carry produced out of the MSB is simply discarded.

Arithmetic Overflow Rule in 2's Complement

Overflow occurs when an addition of two numbers with identical signs produces an invalid result with an opposite sign. Overflow cannot occur when adding numbers with opposite signs.

$$V = C_n \oplus C_{n-1} \tag{5}$$

where C_n is the carry out of the sign bit (MSB), and C_{n-1} is the carry into the sign bit. If $V = 1$, arithmetic overflow has occurred, indicating the result exceeds the allowable n-bit range.

2. Boolean Algebra and Logic Simplification

Boolean algebra is an algebraic structure defined on a set of elements $B = \{0, 1\}$ with two binary operators ($+$ OR, $\cdot$ AND) and one unary operator ($'$ NOT).

2.1 Fundamental Laws of Boolean Algebra

Law / Property	AND Form	OR Form
Identity Law	$A \cdot 1 = A$	$A + 0 = A$
Null (Annulment) Law	$A \cdot 0 = 0$	$A + 1 = 1$
Idempotent Law	$A \cdot A = A$	$A + A = A$
Complement Law	$A \cdot A' = 0$	$A + A' = 1$
Involution (Double Negation)	$(A')' = A$	

© Biraj

Law / Property (contd.)	AND Form	OR Form
Commutative Law	$A \cdot B = B \cdot A$	$A + B = B + A$
Associative Law	$A \cdot (B \cdot C) = (A \cdot B) \cdot C$	$A + (B + C) = (A + B) + C$
Distributive Law	$A \cdot (B + C) = A \cdot B + A \cdot C$	$A + (B \cdot C) = (A + B) \cdot (A + C)$
Absorption Law	$A \cdot (A + B) = A$	$A + A \cdot B = A$
Consensus Theorem	$(A + B)(A' + C)(B + C) = (A + B)(A' + C)$	$AB + A'C + BC = AB + A'C$

2.2 De Morgan's Theorems and the Principle of Duality

De Morgan's Theorems

$$\overline{A \cdot B \cdot C \ldots} = \overline{A} + \overline{B} + \overline{C} + \ldots \quad \text{(NAND gate is equivalent to a Bubbled-OR gate)} \tag{6}$$

$$\overline{A + B + C + \ldots} = \overline{A} \cdot \overline{B} \cdot \overline{C} \ldots \quad \text{(NOR gate is equivalent to a Bubbled-AND gate)} \tag{7}$$

Principle of Duality: Any true Boolean identity remains valid if:

a. Every $+$ (OR) operator is replaced with $\cdot$ (AND), and vice versa.

b. Every 0 is replaced with 1, and vice versa.

c. Literal variables remain uncomplemented.

2.3 Canonical and Standard Forms

- **Minterm (m_i):** A product (AND) of all n literals in which each variable appears exactly once in true or complemented form. A minterm equals 1 for exactly one input combination.
- **Maxterm (M_i):** A sum (OR) of all n literals in which each variable appears exactly once. A maxterm equals 0 for exactly one input combination.
- **Complementary Duality:**

$$M_i = \overline{m_i}, \qquad f(A, B, C) = \sum m(1, 4, 6, 7) = \prod M(0, 2, 3, 5) \tag{8}$$

- **Sum of Products (SOP):** Canonical form summing all minterms where the function is 1.
- **Product of Sums (POS):** Canonical form multiplying all maxterms where the function is 0.

3. Karnaugh Map (K-Map) Simplification

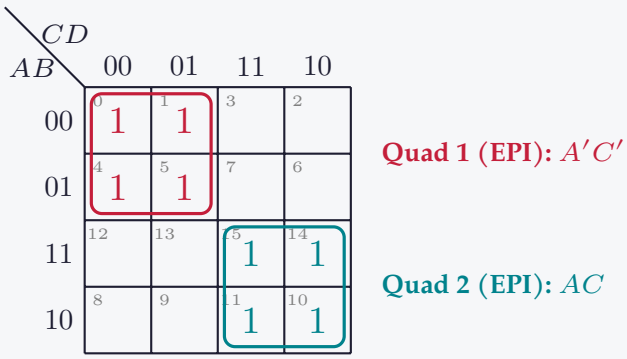
The Karnaugh Map is a graphical truth-table layout where adjacent squares differ in only a single bit position (Gray code ordering: $00, 01, 11, 10$).

© Biraj

3.1 Grouping Principles and Implicants

1. Adjacent squares containing 1s are combined into groups of sizes that are powers of two $(1, 2, 4, 8, 16)$.
2. Grouping 2^k adjacent cells eliminates k variables.
3. **Implicant:** Any single minterm or combination of minterms that can be grouped.
4. **Prime Implicant (PI):** A rectangle of 1s that cannot be merged with any larger group.
5. **Essential Prime Implicant (EPI):** A prime implicant that covers at least one minterm (1) not covered by any other prime implicant. All EPIs must be included in the minimal Boolean expression.
6. **Don't Care Conditions (d or X):** Input combinations that can never occur in practice. They can be treated as 1s if they help enlarge a group, or as 0s if they do not.

4-Variable Karnaugh Map Cell Topology and Adjacency Structure



3.2 Static Hazards in Combinational Logic

A **static hazard** is a transient unwanted glitch ($0 \to 1 \to 0$ for static-0 or $1 \to 0 \to 1$ for static-1) occurring at the output due to unequal propagation delays along converging signal paths.

- **Cause:** A single variable changes state and transitions between two adjacent K-map prime implicant loops without an overlapping product term.
- **Elimination:** Add redundant consensus prime implicants to bridge adjacent loops in the K-map.

4. Binary Codes and Code Converters

Digital systems employ specialized binary codes tailored for arithmetic, display, communication, and mechanical position sensing:

4.1 Standard Binary Codes

- **BCD (Binary Coded Decimal / 8421):** Each decimal digit (0–9) is encoded as a 4-bit binary group (0000 to 1001). The six combinations 1010_2 to 1111_2 are invalid.

© Biraj

- **Excess-3 Code:** Unweighted, self-complementing code formed by adding 0011_2 $(+3)$ to the 8421 BCD code. The 1's complement of an Excess-3 number represents the 9's complement of its decimal equivalent.
- **Gray Code (Reflected Binary Unit-Distance Code):** Successive code words differ in only a single bit position $(\Delta = 1)$, eliminating spurious switching errors in optical shaft encoders.

4.2 Binary to Gray and Gray to Binary Code Conversion

Code Conversion Logic

Binary to Gray conversion $(B = b_{n-1} \dots b_0 \rightarrow G = g_{n-1} \dots g_0)$:

$$g_{n-1} = b_{n-1}, \qquad g_i = b_{i+1} \oplus b_i \quad (\text{for } i = 0, 1, \dots, n-2) \tag{9}$$

Gray to Binary conversion $(G = g_{n-1} \dots g_0 \rightarrow B = b_{n-1} \dots b_0)$:

$$b_{n-1} = g_{n-1}, \qquad b_i = b_{i+1} \oplus g_i \quad (\text{for } i = 0, 1, \dots, n-2) \tag{10}$$

5. Combinational Arithmetic Circuits

5.1 Half Adder, Full Adder, and Subtractors

Circuit	Boolean Sum / Difference (S, D)	Carry / Borrow (C, B)
Half Adder (HA)	$S = A \oplus B$	$C = A \cdot B$
Half Subtractor (HS)	$D = A \oplus B$	$B_{\text{out}} = A' \cdot B$
Full Adder (FA)	$S = A \oplus B \oplus C_{\text{in}}$	$C_{\text{out}} = AB + (A \oplus B)C_{\text{in}} = AB + BC_{\text{in}} + AC_{\text{in}}$
Full Subtractor (FS)	$D = A \oplus B \oplus B_{\text{in}}$	$B_{\text{out}} = A'B + (A \oplus B)'B_{\text{in}} = A'B + A'B_{\text{in}} + BB_{\text{in}}$

5.2 Ripple Carry Adder vs. Carry Lookahead Adder (CLA)

An n-bit **Ripple Carry Adder (RCA)** cascades n Full Adders. The carry out of stage i ripples to the carry input of stage $i+1$. Total worst-case propagation delay is linear:

$$t_{\text{RCA}} = 2t_{\text{gate}} + (n-1)2t_{\text{gate}} = 2n\, t_{\text{gate}} \tag{11}$$

To eliminate rippling delay, the **Carry Lookahead Adder (CLA)** precomputes all stage carries in parallel using Carry Generate (G_i) and Carry Propagate (P_i) functions:

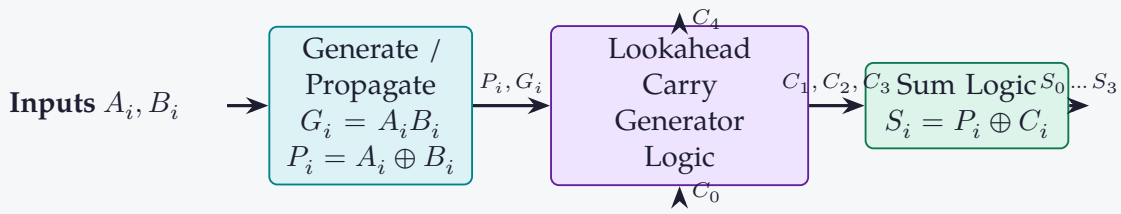
$$G_i = A_i \cdot B_i, \qquad P_i = A_i \oplus B_i \tag{12}$$

© Biraj

Expanding the stage carry expressions recursively:

$$C_1 = G_0 + P_0C_0 \tag{13}$$
$$C_2 = G_1 + P_1C_1 = G_1 + P_1G_0 + P_1P_0C_0 \tag{14}$$
$$C_3 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0 \tag{15}$$
$$C_4 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_0 \tag{16}$$

Carry Lookahead Generator Architecture (4-bit CLA)



Delay Comparison: All carries $C_1, \dots, C_4$ are generated in only $2t_{\text{gate}}$ delays, and the final sums in $4t_{\text{gate}}$ delays, completely independent of word length n.

5.3 BCD Adder

Adding two 4-bit BCD digits produces a sum $S_3S_2S_1S_0$ and carry K. When the sum exceeds 9 (1001_2) or a carry $K = 1$ is generated, the result is an invalid BCD digit. Correction is achieved by adding 6 (0110_2):

$$\text{Correction Flag } C_{\text{out}} = K + S_3S_2 + S_3S_1 \tag{17}$$

6. MSI Combinational Logic Devices

6.1 Magnitude Comparators

A magnitude comparator compares two n-bit binary numbers $A = a_{n-1} \dots a_0$ and $B = b_{n-1} \dots b_0$ and asserts one of three output lines: $A > B$, $A = B$, or $A < B$.

- Equality condition: $A = B \implies x_i = (a_i \oplus b_i)'$ for all i, so $(A = B) = x_{n-1}x_{n-2} \dots x_0$.
- Greater-than condition:

$$(A > B) = a_{n-1}b'_{n-1} + x_{n-1}a_{n-2}b'_{n-2} + \cdots + x_{n-1} \dots x_1a_0b'_0 \tag{18}$$

- IC 7485 is a 4-bit magnitude comparator equipped with cascading inputs $(I_{A>B}, I_{A=B}, I_{A<B})$ enabling expansion to compare 8-bit, 16-bit, or larger words.

6.2 Multiplexers (Data Selectors)

A multiplexer routes one of 2^n data input channels to a single output line under the control of n selection lines:

$$Y = \sum_{k=0}^{2^n-1} m_k \cdot D_k \tag{19}$$

where m_k is the minterm formed by the n select lines $S_{n-1} \dots S_0$.

© Biraj

- **Universal Logic Implementation:** Any Boolean function of n variables can be implemented directly using a 2^{n-1}-to-1 MUX by connecting $(n-1)$ variables to select lines and using the remaining variable $(0, 1, X, X')$ on the data inputs.

6.3 Encoders and Decoders

- **Decoders (n-to-2^n):** Converts an n-bit binary input into one of 2^n active output lines (minterm generator). IC 74138 is a standard 3-to-8 decoder with active-low outputs.
- **Priority Encoders (2^n-to-n):** If two or more inputs are simultaneously asserted, the encoder generates the binary code corresponding to the input with the highest assigned priority (e.g., IC 74148 8-to-3 priority encoder).

6.4 Barrel Shifters and Arithmetic Logic Units (ALUs)

- **Barrel Shifter:** A specialized digital crossbar circuit constructed from multiplexer arrays capable of shifting or rotating an n-bit data word by an arbitrary number of bit positions (0 to $n-1$) in a **single clock cycle**, avoiding sequential shift register clock cycles.
- **Arithmetic Logic Unit (IC 74181):** A 4-bit parallel multifunction processor element. Under the command of 4 function select inputs ($S_3S_2S_1S_0$) and a mode control bit M ($M = 1 \implies$ Logic functions; $M = 0 \implies$ Arithmetic functions with carry lookahead), it can perform all 16 logic operations and 16 arithmetic operations.

© Biraj

7. Quick Revision Page

Key Formulas and Identities at a Glance

- **Signed 2's Complement Range:** $-2^{n-1} \leq N \leq 2^{n-1} - 1$
- **2's Complement Overflow Condition:** $V = C_n \oplus C_{n-1}$
- **De Morgan's Laws:** $\overline{A \cdot B} = \overline{A} + \overline{B}, \quad \overline{A + B} = \overline{A} \cdot \overline{B}$
- **Binary-to-Gray Conversion:** $g_{n-1} = b_{n-1}, \quad g_i = b_{i+1} \oplus b_i$
- **Full Adder Sum & Carry:** $S = A \oplus B \oplus C_{\text{in}}, \quad C_{\text{out}} = AB + (A \oplus B)C_{\text{in}}$
- **CLA Generate & Propagate:** $G_i = A_i B_i, \quad P_i = A_i \oplus B_i, \quad C_{i+1} = G_i + P_i C_i$
- **BCD Adder Correction:** $C_{\text{out}} = K + S_3 S_2 + S_3 S_1$
- **Multiplexer Output Equation:** $Y = \sum_{k=0}^{2^n-1} m_k D_k$

One-Line Definitions

- **Minterm:** A canonical product term of n variables that evaluates to logic 1 for exactly one input combination.
- **Essential Prime Implicant (EPI):** A prime implicant rectangle containing at least one minterm that is not covered by any other prime implicant.
- **Static-1 Hazard:** A transient output glitch to 0 occurring when an input change should have maintained a constant 1 output.
- **Gray Code:** An unweighted, unit-distance binary code in which adjacent numbers differ by exactly one bit.
- **Priority Encoder:** An encoder that outputs the binary address of only the highest-priority active input line.
- **Barrel Shifter:** A combinational logic circuit that shifts or rotates an n-bit word by any specified number of bits in a single clock cycle.

Frequently Asked / Viva Questions

Q1. Why is 2's complement universally preferred over 1's complement in digital ALUs?

2's complement provides a single unique representation for zero (0000_2), eliminates the complex end-around carry addition required in 1's complement, and simplifies hardware subtraction into simple addition.

Q2. How do you detect arithmetic overflow in 2's complement addition?

Overflow occurs when adding two numbers of identical sign results in a sum with the opposite sign. In hardware, it is detected when the carry into the sign bit differs from the carry out of the sign bit ($V = C_n \oplus C_{n-1} = 1$).

© Biraj

Frequently Asked / Viva Questions (contd.)

Q3. What is the difference between Prime Implicants and Essential Prime Implicants in a K-map?

A Prime Implicant is a maximal group of adjacent 1s that cannot be doubled in size. An Essential Prime Implicant is a prime implicant that covers at least one 1 that is not covered by any other prime implicant.

Q4. Explain the cause and remedy of a static-1 hazard.

It is caused by unequal propagation delays when a variable and its complement change simultaneously across two separate K-map loops. It is remedied by adding an extra overlapping consensus prime implicant to cover the transition.

Q5. Why is Gray code extensively used in optical shaft encoders?

Because only a single bit changes between any two consecutive values ($\Delta = 1$), avoiding erroneous transient readings caused by mechanical switch misalignment.

Q6. Why is a Carry Lookahead Adder (CLA) faster than a Ripple Carry Adder (RCA)?

In an RCA, carry must propagate sequentially through all n stages ($t \propto n$). In a CLA, carries are computed simultaneously in two gate delays using carry generate (G_i) and propagate (P_i) logic ($t = O(1)$).

Q7. What is the correction condition in a BCD adder?

When the binary sum of two BCD digits exceeds 9 (1001_2) or generates an output carry $K = 1$, the correction logic $C = K + S_3S_2 + S_3S_1$ adds $+6$ (0110_2) to skip the 6 invalid 4-bit states.

Q8. How can an n-variable Boolean function be implemented using an 2^{n-1}-to-1 MUX?

Connect $(n-1)$ variables to the select lines, and connect the data input lines to $0, 1$, or the remaining variable X or X' derived from a truth table partition.

Q9. What is a Priority Encoder and where is it used?

It is an encoder that produces the binary code corresponding to the highest-order active input when multiple inputs are asserted. It is widely used in microprocessor interrupt request handling.

Q10. What is the function of the mode control bit M in the 74181 ALU?

$M = 1$ disables internal carry propagation, configuring the ALU for 16 logic operations. $M = 0$ enables carries, configuring the ALU for 16 arithmetic operations.

Q11. Differentiate between canonical and standard Boolean expressions.

In canonical form (canonical SOP/POS), every individual product/sum term must contain all variables of the function. In standard form, terms may contain any subset of variables.

© Biraj

Frequently Asked / Viva Questions (contd.)

Q12. What is a barrel shifter and why is it preferred over a shift register in microprocessors?

A barrel shifter uses multiplexer matrices to shift or rotate an n-bit word by k positions in a single clock cycle, whereas an ordinary shift register requires k separate clock cycles.

Comparison of Combinational MSI Building Blocks

Device	Input / Output Ratio	Control Signals	Primary Application
Multiplexer (MUX)	2^n Data Inputs $\rightarrow$ 1 Output	n Select Lines, Enable	Data routing, parallel-to-serial conversion, Boolean function synthesis
Demultiplexer (DEMUX)	1 Data Input $\rightarrow$ 2^n Outputs	n Select Lines, Enable	Serial-to-parallel distribution, memory addressing
Decoder	n Inputs $\rightarrow$ 2^n Outputs	Enable / Chip Select	Memory address decoding, 7-segment displays, minterm generation
Priority Encoder	2^n Inputs $\rightarrow$ n Outputs	Enable, Group Select	Interrupt arbitration, ADC flash encoding
Magnitude Comparator	$2 \times n$ Bit Inputs $\rightarrow$ 3 Outputs	Cascading Inputs $(>, =, <)$	Arithmetic sorting, threshold checking, address limit checks

© Biraj

Module 2
Latches and Flip-Flops

1. Latches and Flip-Flops

In sequential logic circuits, outputs depend not only on current inputs but also on past history stored in memory elements.

1.1 Latches vs. Flip-Flops

- **Latch:** A level-triggered bistable storage device that responds continuously to inputs whenever the enable/clock signal is at an active level (transparent).
- **Flip-Flop:** An edge-triggered storage device that samples inputs and updates outputs only at a discrete transition (positive rising edge ↑ or negative falling edge ↓) of the clock signal.
- **Setup Time (t_s):** The minimum time for which the data input must remain stable *before* the arrival of the active clock edge.
- **Hold Time (t_h):** The minimum time for which the data input must remain stable *after* the active clock edge.
- **Metastability:** An unstable intermediate output voltage state occurring if setup or hold times are violated.

1.2 The S-R Latch and Forbidden State

An S-R (Set-Reset) latch constructed with cross-coupled NOR gates operates as follows:

- $S = 0, R = 0 \implies Q_{n+1} = Q_n$ (No change / Hold state).
- $S = 0, R = 1 \implies Q_{n+1} = 0, \overline{Q}_{n+1} = 1$ (Reset state).
- $S = 1, R = 0 \implies Q_{n+1} = 1, \overline{Q}_{n+1} = 0$ (Set state).
- $S = 1, R = 1 \implies Q = \overline{Q} = 0$ (**Invalid / Forbidden State**). When both inputs simultaneously drop from 1 to 0, the circuit enters a race condition, settling into an unpredictable state.

1.3 J-K Flip-Flop and the Race-Around Condition

The J-K flip-flop eliminates the invalid state of the S-R flip-flop by feeding back outputs Q and $\overline{Q}$ to the input AND gates:

$$S = J \cdot \overline{Q}, \qquad R = K \cdot Q \tag{20}$$

When $J = K = 1$, the flip-flop toggles its output on each active clock: $Q_{n+1} = \overline{Q}_n$.

© Biraj

The Race-Around Condition in Level-Triggered JK Flip-Flops

Definition: In a level-triggered JK flip-flop, when $J = K = 1$ and the clock pulse width t_w is greater than the propagation delay t_{pd} of the flip-flop ($t_{pd} < t_w$), the output toggles repeatedly and uncontrollably back and forth ($0 \to 1 \to 0 \to 1 \ldots$) during the single high clock interval. At the end of the clock pulse, the final state of Q is indeterminate.
Mathematical Condition for Race-Around:

$$t_{pd} < t_w < \frac{T}{2} \tag{21}$$

where T is the clock period and t_w is the pulse duration.
Methods to Prevent Race-Around:

a. **Edge Triggering:** Design the circuit to respond only to steep clock transitions ($t_w \approx 0$).

b. **Propagation Delay Control:** Choose components such that $t_{pd} > t_w$ (difficult and limits maximum operating frequency).

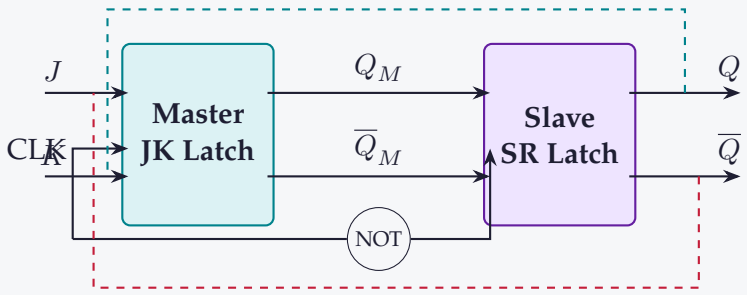
c. **Master-Slave JK Flip-Flop Configuration:** The universal and standard architecture.

1.4 Master-Slave JK Flip-Flop Architecture

The Master-Slave JK flip-flop consists of two cascaded latches clocked on complementary phases:

- **Master Latch:** Enabled when CLK $= 1$. It samples inputs J and K and updates its intermediate state (Q_M). The Slave latch remains disabled (isolated) because $\overline{\text{CLK}} = 0$.
- **Slave Latch:** When the clock falls ($\text{CLK} = 0 \implies \overline{\text{CLK}} = 1$), the Master is disabled, locking its state. The Slave is enabled and copies the state Q_M to the final external output Q.
- Because the output Q updates only on the clock falling edge when the Master is completely disabled, feedback cannot affect the Master during the same clock cycle, **completely eliminating race-around**.

Master-Slave JK Flip-Flop Functional Architecture



© Biraj

1.5 Characteristic Equations and Excitation Tables

Type	Characteristic Equation	$Q_n \to Q_{n+1}$: $0 \to 0$	$0 \to 1$	$1 \to 0$	$1 \to 1$
S-R	$Q_{n+1} = S + \overline{R}Q_n$ $(SR = 0)$	$S = 0, R = X$	$S = 1, R = 0$	$S = 0, R = 1$	$S = X, R = 0$
J-K	$Q_{n+1} = J\overline{Q}_n + \overline{K}Q_n$	$J = 0, K = X$	$J = 1, K = X$	$J = X, K = 1$	$J = X, K = 0$
D	$Q_{n+1} = D$	$D = 0$	$D = 1$	$D = 0$	$D = 1$
T	$Q_{n+1} = T \oplus Q_n$	$T = 0$	$T = 1$	$T = 1$	$T = 0$

2. Registers and Shift Registers

An n-bit register is a cascade of n flip-flops sharing a common clock, designed to store and shift multi-bit binary data words.

2.1 Shift Register Configurations

- **SISO (Serial-In Serial-Out):** Data enters one bit per clock cycle and exits serially. Requires n clock cycles to load and n clock cycles to read an n-bit word.
- **SIPO (Serial-In Parallel-Out):** Data enters serially; all n bits are accessible simultaneously on parallel outputs after n clock cycles.
- **PISO (Parallel-In Serial-Out):** Data is loaded into all flip-flops simultaneously in a single clock cycle using parallel load gates, then shifted out serially.
- **PIPO (Parallel-In Parallel-Out):** Parallel loading and parallel output reading simultaneously in 1 clock cycle.

2.2 Universal Shift Register (IC 74194)

A bidirectional 4-bit Universal Shift Register can perform all four data transfer operations controlled by two mode select inputs S_1, S_0:

$$S_1S_0 = 00 \implies \text{Hold}, \quad 01 \implies \text{Shift Right}, \quad 10 \implies \text{Shift Left}, \quad 11 \implies \text{Parallel Load} \tag{22}$$

3. Asynchronous (Ripple) and Synchronous Counters

A counter is a specialized sequential circuit that cycles through a prescribed sequence of binary states upon receiving clock pulses.

© Biraj

3.1 Asynchronous (Ripple) Counters

In a ripple counter, the external clock is applied **only to the first flip-flop (LSB)**. Each subsequent flip-flop is clocked by the output of the preceding stage (Q or $\overline{Q}$).

- **Up-Counter vs. Down-Counter:**
  - Negative edge-triggered FF with clock connected to $Q_i \implies$ Up-counter.
  - Negative edge-triggered FF with clock connected to $\overline{Q}_i \implies$ Down-counter.
- **Mod-N Ripple Counter:** Formed by resetting all flip-flops using an active-low asynchronous clear ($\overline{\text{CLR}}$) driven by a NAND gate when the count reaches state N.
- **Disadvantages of Ripple Counters:**
  1. Accumulated delay: Total propagation delay is $t_{\text{total}} = n \times t_{pd}$.
  2. Maximum operating frequency is constrained: $f_{\max} \leq \frac{1}{n \cdot t_{pd}}$.
  3. Transient false counting states (glitches) appear on outputs during transition intervals.

3.2 Synchronous Counters

In a synchronous counter, the master clock is connected **in parallel to all flip-flops simultaneously**. State transitions occur at the exact same clock instant.

Systematic Synchronous Counter Design Procedure:

1. Determine number of flip-flops: $2^n \geq N$.
2. Construct the State Transition Diagram and State Table.
3. Substitute flip-flop excitation requirements (J_i, K_i or T_i or D_i) from the excitation table.
4. Minimize excitation expressions using Karnaugh maps.
5. Implement the logic circuit.

3.3 Ring Counter and Johnson (Twisted Ring) Counter

- **Ring Counter:** An n-bit shift register with true output Q_n fed back to serial input D_1. A single circulating '1' ($1000 \to 0100 \to 0010 \to 0001$).

$$\text{Number of unique states} = n, \qquad \text{Unused states} = 2^n - n \tag{23}$$

- **Johnson (Twisted Ring) Counter:** An n-bit shift register with inverted output $\overline{Q}_n$ fed back to D_1.

$$\text{Number of unique states} = 2n, \qquad \text{Unused states} = 2^n - 2n \tag{24}$$

  Requires only simple 2-input AND gates for complete glitch-free state decoding.

4. Finite State Machines (FSM)

Any synchronous sequential circuit can be modeled formally as a Finite State Machine consisting of combinational logic and state memory registers.

© Biraj

4.1 Mealy vs. Moore FSM Models

Architectural Definitions of Mealy and Moore Machines

- **Moore Machine:** The output Z depends **only on the present state** S:

$$Z(t) = \lambda(S(t)) \tag{25}$$

  Outputs are synchronous with the clock and immune to input glitches.

- **Mealy Machine:** The output Z depends on **both the present state** S **and current inputs** X:

$$Z(t) = \lambda(S(t), X(t)) \tag{26}$$

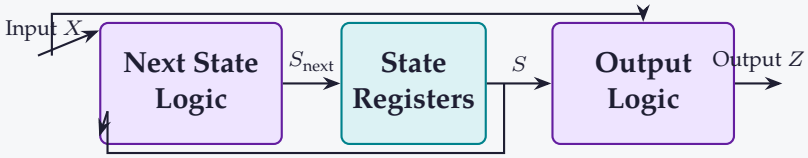
  Outputs change asynchronously as soon as inputs change, enabling faster response and generally requiring fewer states (1 state fewer than Moore).

Block Diagrams: Moore Machine vs. Mealy Machine

Moore Machine: $Z = \lambda(S)$



Mealy Machine: $Z = \lambda(S, X)$



4.2 State Reduction and State Assignment

- **Equivalent States:** Two states S_1 and S_2 are equivalent if for every possible input sequence, they produce identical outputs and transition to identical (or equivalent) next states.
- **State Reduction:** Performed using row elimination or the **Implication Table method** to eliminate redundant states, directly minimizing flip-flop count.
- **State Assignment Techniques:**
  - **Binary Encoding:** Uses minimum number of flip-flops ($n = \lceil \log_2 N \rceil$), requires more combinational decode logic.
  - **Gray Encoding:** Consecutive states differ by 1 bit, minimizing switching noise and power.
  - **One-Hot Encoding:** Uses 1 flip-flop per state (N flip-flops for N states). Maximizes speed and simplifies next-state decode logic; standard choice in FPGA designs.

5. Design of Synchronous Circuits

© Biraj

5.1 Sequence Detector / Pulse Train Generator

A sequence detector inspects an incoming serial binary bit stream and asserts output $Z = 1$ whenever a specific target pattern (e.g., 1011) is recognized.

- **Overlapping Sequence Detection:** The final bit(s) of a detected sequence can serve as the prefix of the next sequence. For 1011 with input 1011011 ... , two detections occur.
- **Non-Overlapping Sequence Detection:** The detector resets to the initial state upon finding the sequence; subsequent detection starts afresh.

5.2 Pseudo-Random Binary Sequence (PRBS) Generator

A PRBS generator produces a deterministic, periodic binary sequence possessing noise-like statistical properties using a **Linear Feedback Shift Register (LFSR)**.

- Constructed from an n-stage shift register where the input to the first flip-flop is the XOR sum of specific tap outputs defined by a primitive polynomial:

$$P(X) = X^n + c_{n-1}X^{n-1} + \cdots + c_1 X + 1 \tag{27}$$

- **Maximal-Length Sequence (m-sequence):** When $P(X)$ is primitive, the LFSR cycles through all possible non-zero states:

$$L = 2^n - 1 \tag{28}$$

  (The all-zero state 00 ... 0 must be avoided because XOR$(0, 0) = 0$, causing lock-up).

- **Key Applications:** Spread-spectrum communications (CDMA PN codes), cryptographic stream ciphers, and Built-In Self-Test (BIST) pseudo-random test pattern generators.

© Biraj

6. Quick Revision Page

Key Sequential Formulas and Equations at a Glance

- **SR Flip-Flop Equation:** $Q_{n+1} = S + \overline{R}Q_n \quad (S \cdot R = 0)$
- **JK Flip-Flop Equation:** $Q_{n+1} = J\overline{Q}_n + \overline{K}Q_n$
- **D Flip-Flop Equation:** $Q_{n+1} = D$
- **T Flip-Flop Equation:** $Q_{n+1} = T \oplus Q_n$
- **Race-Around Condition:** $t_{pd} < t_w < T/2 \quad (J = K = 1)$
- **Ripple Counter Maximum Delay:** $t_{\text{total}} = n \cdot t_{pd} \implies f_{\max} = 1/(n \cdot t_{pd})$
- **Ring Counter States:** $N = n \quad (n\text{-bit shift register})$
- **Johnson Counter States:** $N = 2n \quad (n\text{-bit shift register})$
- **Maximal LFSR / PRBS Period:** $L = 2^n - 1$

One-Line Definitions

- **Setup Time (t_s):** Minimum interval data must remain stable before the active clock edge.
- **Hold Time (t_h):** Minimum interval data must remain stable after the active clock edge.
- **Race-Around Condition:** Repeated uncontrolled toggling of a level-triggered JK flip-flop during clock high when $t_{pd} < t_w$.
- **Moore Machine:** A finite state machine whose output is strictly a function of current state only.
- **Mealy Machine:** A finite state machine whose output depends simultaneously on current state and current inputs.
- **Johnson Counter:** An inverted feedback shift register producing $2n$ states for n flip-flops.

Frequently Asked / Viva Questions

Q1. What is the difference between a latch and a flip-flop?

A latch is level-triggered (transparent whenever clock/enable is active), whereas a flip-flop is edge-triggered (samples and locks data strictly on clock transition edges).

Q2. Explain the race-around condition in a JK flip-flop. How is it eliminated?

When $J = K = 1$ in a level-triggered JK FF, if pulse width $t_w > t_{pd}$, the output toggles multiple times before the clock drops, leaving an uncertain final state. It is eliminated by using edge triggering or a Master-Slave configuration.

© Biraj

Frequently Asked / Viva Questions (contd.)

Q3. Why is Master-Slave JK flip-flop pulse-triggered rather than edge-triggered?

Because the master accepts data while the clock is high, and data transfers to the slave on the falling edge; output changes only once per complete clock period.

Q4. How do you convert a JK flip-flop into a D flip-flop?

Connect input D directly to J, and connect an inverted D ($\overline{D}$) to K ($J = D, K = \overline{D}$).

Q5. What is the propagation delay limitation of ripple counters?

Carries must ripple through each stage sequentially ($t = n \cdot t_{pd}$), which limits the maximum clock frequency and creates transient false decoding spikes (glitches).

Q6. Compare Ring counter and Johnson counter for an n-bit shift register.

A ring counter yields n states and requires no decoding gates. A Johnson counter yields $2n$ states and requires simple 2-input AND gates for decoding.

Q7. What are the key architectural differences between Mealy and Moore FSMs?

In a Moore machine, output depends solely on current state ($Z = \lambda(S)$); in a Mealy machine, output depends on both current state and inputs ($Z = \lambda(S, X)$).

Q8. Why does a Mealy machine typically require fewer states than a Moore machine?

Because Mealy outputs can vary immediately within a state based on different inputs, whereas a Moore machine requires distinct states for distinct output combinations.

Q9. What is meant by One-Hot state assignment?

An encoding scheme using N flip-flops for N states where exactly one flip-flop is high ('1') at any time. It minimizes combinational decoding delay and is optimal for FPGAs.

Q10. What is an LFSR and why can the all-zero state not be used?

A Linear Feedback Shift Register uses XOR taps to generate pseudo-random sequences. If all flip-flops become 0, $\text{XOR}(0, 0) = 0$, locking the register in permanent zeros.

Q11. What is metastability in digital circuits?

A condition where a flip-flop's output hovers at an undefined intermediate voltage level between logic 0 and 1 for an indeterminate duration due to a setup/hold time violation.

Q12. How does an overlapping sequence detector differ from a non-overlapping one?

An overlapping detector allows the trailing bits of a detected sequence to be reused as the leading bits of the subsequent sequence, producing more output pulses.

© Biraj

Comparison: Ripple vs. Synchronous Counters and Mealy vs. Moore FSM

Category	Architecture A	Architecture B	Key Design Trade-off
Counter Clocking	**Ripple Counter:** Flip-flops clocked by previous output	**Synchronous Counter:** All flip-flops clocked simultaneously	Ripple uses simpler wiring but has high cumulative delay; Synchronous achieves maximum speed.
FSM Output Logic	**Moore Machine:** $Z = \lambda(S)$ (State only)	**Mealy Machine:** $Z = \lambda(S, X)$ (State & Input)	Moore is glitch-free and simpler; Mealy responds 1 cycle faster with fewer states.
Shift Register Counting	**Ring Counter:** $N = n$ states, uncomplemented loop	**Johnson Counter:** $N = 2n$ states, inverted loop	Johnson provides double the count states with minimal decoding hardware.

© Biraj

Module 3

Digital Logic Families and Performance Metrics

1. Digital Logic Families and Performance Metrics

Digital Integrated Circuits (ICs) are fabricated using distinct semiconductor device technologies categorized into bipolar and metal-oxide-semiconductor (MOS) technologies.

1.1 Key Performance Metrics of Digital ICs

Core Figures of Merit for Logic Families

- **Propagation Delay (t_{pd}):** The finite time required for a signal to transition through a gate from input to output, defined as the average of low-to-high (t_{PLH}) and high-to-low (t_{PHL}) transition times measured at the 50% amplitude points:

$$t_{pd} = \frac{t_{PLH} + t_{PHL}}{2} \tag{29}$$

- **Power Dissipation (P_D):** The electrical power supplied by the DC source to the gate:

$$P_D = I_{CC(\text{avg})} \times V_{CC} = \left(\frac{I_{CCH} + I_{CCL}}{2}\right) \times V_{CC} \tag{30}$$

- **Power-Delay Product (PDP):** The fundamental figure of merit reflecting the speed-power trade-off (expressed in Joules or picojoules, pJ):

$$\text{PDP} = P_D \times t_{pd} \tag{31}$$

  A smaller PDP denotes a superior, highly efficient logic family.

© Biraj

- **Noise Margins (NM_H, NM_L):** The maximum spurious noise voltage amplitude that can be superimposed on an input logic signal without corrupting the recognized output state:

$$\text{High-State Noise Margin:} \quad NM_H = V_{OH(\text{min})} - V_{IH(\text{min})} \tag{32}$$

$$\text{Low-State Noise Margin:} \quad NM_L = V_{IL(\text{max})} - V_{OL(\text{max})} \tag{33}$$

- **Fan-Out (N):** The maximum number of standard load inputs of the same family that a gate output can reliably drive without degrading logic levels:

$$N = \min\left(\left\lfloor \frac{I_{OH(\text{max})}}{I_{IH(\text{max})}} \right\rfloor, \left\lfloor \frac{I_{OL(\text{max})}}{I_{IL(\text{max})}} \right\rfloor\right) \tag{34}$$

- **Fan-In:** The total number of independent physical input lines available on a logic gate.

1.2 TTL (Transistor-Transistor Logic)

TTL uses saturated BJT switches operating at a nominal supply voltage of $V_{CC} = 5.0$ V.

- **Multi-Emitter Input BJT (Q_1):** Serves as the input AND gate. When any input is LOW (0.2 V), Q_1 conducts heavily, draining base current away from the phase splitter (Q_2), forcing Q_2 and output pull-down (Q_4) to turn OFF.
- **Phase Splitter Transistor (Q_2):** Produces complementary collector and emitter voltages to drive the push-pull output stage.
- **Totem-Pole Output Stage:** Consists of an active pull-up BJT (Q_3), series protection diode (D_1), and pull-down BJT (Q_4).
  - *Advantage:* Provides active low output impedance during both transitions ($0 \to 1$ and $1 \to 0$), rapidly charging and discharging parasitic capacitive loads.
  - *Critical Rule:* **Totem-pole outputs must NEVER be tied together directly in a wire-AND configuration.** If one gate output attempts to pull HIGH while another pulls LOW, a destructive direct low-resistance short-circuit path from V_{CC} to ground occurs, causing excessive current spikes and thermal destruction.
- **Open-Collector (OC) TTL:** Omits internal pull-up transistor Q_3 and diode D_1, leaving the collector of Q_4 floating.
  - An external pull-up resistor R_P tied to V_{CC} is required.
  - Permits **Wired-AND** logic: Multiple OC collectors can be tied together to a single node. The node is HIGH only if all connected gates turn OFF their pull-down transistors.
  - **Pull-Up Resistor Calculation:**

$$R_{P(\text{max})} = \frac{V_{CC} - V_{OH(\text{min})}}{n \cdot I_{OH(\text{leak})} + N \cdot I_{IH(\text{max})}} \tag{35}$$

$$R_{P(\text{min})} = \frac{V_{CC} - V_{OL(\text{max})}}{I_{OL(\text{max})} - N \cdot I_{IL(\text{max})}} \tag{36}$$

© Biraj

- **Tri-State Logic (TSL):** Features three distinct electrical output states: Logic HIGH, Logic LOW, and **High-Impedance (Hi-Z)**. When the gate is disabled, both totem-pole transistors are simultaneously turned OFF, electrically isolating the node from the shared bus.

1.3 ECL (Emitter-Coupled Logic)

ECL operates on a **non-saturating BJT current-steering** differential pair.

- **Non-Saturating Operation:** Transistors operate strictly in the active and cutoff regions. By preventing BJTs from entering deep saturation, the base minority carrier storage time (t_s) is completely eliminated.
- **Speed:** Fastest bipolar family ($t_{pd} \leq 1$ ns).
- **Power Supply:** Operates with a negative supply voltage ($V_{EE} = -5.2$ V, collector ground $V_{CC} = 0$ V) to protect noise immunity against power supply surges.
- **Complementary Outputs:** Provides true OR and NOR outputs simultaneously.
- **Disadvantages:** Extremely high continuous static power dissipation ($P_D \approx$ 25–40 mW/gate); very small noise margins ($NM \approx 0.2$–0.25 V).

1.4 CMOS Logic Family

CMOS (Complementary Metal-Oxide-Semiconductor) circuits combine complementary pairs of enhancement-mode PMOS and NMOS transistors.

- **Structure:** Pull-Up Network (PUN) composed of PMOS transistors connected to V_{DD}, and Pull-Down Network (PDN) composed of NMOS transistors connected to V_{SS} (GND). For any valid input combination, exactly one network conducts while the other remains completely OFF.
- **CMOS Inverter VTC:** Features five regions:
  - *Region A* ($V_{in} < V_{tn}$)*:* NMOS OFF, PMOS linear ($V_{out} = V_{DD}$).
  - *Region B* ($V_{tn} \leq V_{in} < V_{th}$)*:* NMOS saturated, PMOS linear.
  - *Region C* ($V_{in} = V_{th}$)*:* Both NMOS and PMOS in saturation (steepest transition).
  - *Region D* ($V_{th} < V_{in} \leq V_{DD} - |V_{tp}|$)*:* NMOS linear, PMOS saturated.
  - *Region E* ($V_{in} > V_{DD} - |V_{tp}|$)*:* NMOS linear, PMOS OFF ($V_{out} = 0$ V).
- **Transistor Sizing:** Because electron mobility is higher than hole mobility ($\mu_n \approx 2.5\mu_p$), the PMOS width must be designed approximately 2.5 times wider than the NMOS width:

$$\left(\frac{W}{L}\right)_p \approx 2.5 \left(\frac{W}{L}\right)_n \implies V_{th} = \frac{V_{DD}}{2} \tag{37}$$

- **Power Dissipation in CMOS:**
  - *Static Power:* Negligible leakage current ($I_{\text{leak}} \times V_{DD} \approx$ nW).
  - *Dynamic Power:* Dominates during high-frequency switching when charging/discharging load capacitance C_L:

$$P_{\text{dyn}} = \alpha \cdot C_L \cdot V_{DD}^2 \cdot f \tag{38}$$

where α is the switching activity factor ($0 \leq \alpha \leq 1$), and f is clock frequency.

© Biraj

- **CMOS Transmission Gate (TG):** A parallel combination of NMOS and PMOS controlled by complementary gating signals $(C, \overline{C})$. Passes the entire voltage range from 0 to V_{DD} without suffering threshold voltage drops $(V_{tn}, |V_{tp}|)$.

Parameter	Standard TTL (74xx)	Schottky TTL (74S)	ECL (10K)	CMOS (74HC)
Device Type	Bipolar (saturated)	Bipolar (Schottky clamped)	Bipolar (non-saturated)	MOSFET (CMOS)
Supply Voltage (V_{CC})	$+5.0$ V $\pm 5\%$	$+5.0$ V $\pm 5\%$	-5.2 V	$+2.0$ V to $+6.0$ V
Propagation Delay (t_{pd})	10 ns	3 ns	1 ns (Fastest)	8 ns (at 5 V)
Power Dissipation (P_D)	10 mW/gate	19 mW/gate	25–40 mW/gate	10 nW (Static)

Parameter (contd.)	Standard TTL (74xx)	Schottky TTL (74S)	ECL (10K)	CMOS (74HC)
Power-Delay Product	100 pJ	57 pJ	25–40 pJ	< 1 pJ (Lowest)
Noise Margin (NM)	0.4 V	0.3 V	0.2–0.25 V	0.3–0.45 $\times V_{DD}$ (High)
Fan-Out	10	10	25	> 50 (Static limit ∞)

2. Semiconductor Memories

Semiconductor memories are organized as arrays of $2^k \times m$ bits, where k is the number of address lines (2^k addressable memory words) and m is the data word length.

2.1 Random Access Memory: SRAM vs. DRAM

- **SRAM (Static RAM):**
  - **Cell Structure:** Consists of a **6-Transistor (6T) CMOS cell**. Two cross-coupled CMOS inverters form a bi-stable latch storing true (Q) and complementary $(\overline{Q})$ nodes, accessed via two NMOS pass transistors activated by the Word Line (WL).
  - **Operation:** When WL $= 1$, access transistors connect storage nodes to Bit Lines (BL and $\overline{BL}$). Sensed by high-speed differential sense amplifiers.

© Biraj

  - **Characteristics:** Fast access times (1–10 ns), no periodic refresh required, retained as long as DC power is present.
  - **Drawback:** Requires 6 transistors per bit cell; low packaging density, high silicon area, expensive. Used primarily for Level 1/2/3 CPU cache memory.

- **DRAM (Dynamic RAM):**
  - **Cell Structure:** Consists of a minimal **1-Transistor 1-Capacitor (1T-1C) cell**. Binary data is stored as electrostatic charge on a tiny trench or stacked MOS capacitor ($C_s \approx$ 20–40 fF).
  - **Refresh Cycle:** Sub-threshold junction leakage discharges the capacitor over time. The entire memory must be periodically read and rewritten (**refreshed every few milliseconds**) by dedicated refresh controllers.
  - **Destructive Read:** Reading transfers the tiny capacitor charge onto the bit line capacitance, altering the stored value. Every read cycle must automatically be followed by an internal precharge and write-back restoration cycle.
  - **Characteristics:** Highest packing density, lowest cost per bit, but slower access time (30–60 ns). Standard choice for computer main memory (DDR4/DDR5 SDRAM).

Feature	Static RAM (SRAM)	Dynamic RAM (DRAM)
Basic Storage Cell	6 Transistors (Cross-coupled inverters)	1 Transistor + 1 Storage Capacitor (1T-1C)
Storage Principle	Bi-stable latch state	Electrostatic charge on capacitor
Refresh Required?	No	Yes (periodically every 2–64 ms)
Read Operation	Non-destructive	Destructive (requires write-back)
Access Speed	Extremely Fast (1–10 ns)	Moderate (30–60 ns)
Density / Cell Area	Low density (large cell area)	Extremely High density (minimal cell area)
Power Consumption	Low static, higher dynamic	Low idle, power consumed during refresh
Primary Applications	CPU L1/L2/L3 Cache, register files	Main system RAM (PC, servers, smartphones)

2.2 Read-Only Memories (ROM) and Flash Memory

- **Mask ROM:** Data programmed permanently during IC semiconductor fabrication using customized metallization masks. Zero reprogramming capability; economical only in vast production volumes.
- **PROM (Programmable ROM):** Shipped with all bits containing intact fuses or antifuses. The user permanently programs the chip using high-voltage current pulses that blow selected fuses. One-time programmable (OTP).

© Biraj

- **EPROM (Erasable PROM):** Uses Floating-Gate Avalanche-injection MOS (FAMOS) transistors. Electrons injected into isolated polysilicon floating gates alter threshold voltages. Erased completely by exposing the die to strong **Ultraviolet (UV) radiation** (253.7 nm) through a transparent quartz window for 15–20 minutes. Cannot be erased in-circuit or selectively byte-erased.
- **EEPROM (Electrically Erasable PROM):** Utilizes Floating Gate Tunnel Oxide (FLOTOX) structures. Electrons are transferred to/from floating gates via **Fowler-Nordheim quantum mechanical tunneling**. Can be erased and rewritten electrically byte-by-byte in-circuit without UV light.
- **Flash Memory:** Advanced high-density EEPROM with block-level erase capability:
  - **NOR Flash:** Storage cells connected in parallel. Supports random access and execute-in-place (XIP) memory architectures. Ideal for storing microcontroller firmware and BIOS code.
  - **NAND Flash:** Storage cells connected in series strings. Provides significantly higher silicon packing density, faster write and erase cycle times, and lower cost per gigabyte. Standard technology for USB flash drives, memory cards, and Solid-State Drives (SSDs).

3. Programmable Logic Devices (PLDs) and FPGAs

Programmable Logic Devices are standard ICs containing reconfigurable arrays of logic gates and storage elements that allow custom digital circuits to be implemented post-fabrication.

3.1 Classification of Simple PLDs (SPLDs)

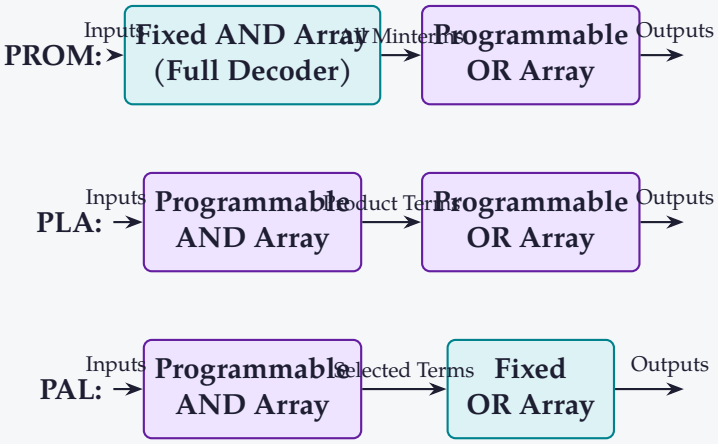
Architectural Comparison of PROM, PLA, and PAL

- **PROM as a PLD:**
  - **AND Array:** Fixed (configured as a complete k-to-2^k binary decoder generating all 2^k minterms).
  - **OR Array:** Fully Programmable.
  - *Limitation:* Fixed full decoding wastes chip area when functions require only a small subset of minterms.
- **PLA (Programmable Logic Array):**
  - **AND Array:** Fully Programmable.
  - **OR Array:** Fully Programmable.
  - *Advantage:* Maximum functional flexibility; allows sharing of product terms among multiple output functions.
  - *Drawback:* Higher internal propagation delay and complex programming hardware.
- **PAL (Programmable Array Logic):**

© Biraj

- **AND Array:** Fully Programmable.
- **OR Array:** Fixed (each OR gate is hardwired to a predetermined set of AND gate outputs).
- *Advantage:* Faster propagation speed, simpler fabrication, and lower cost compared to PLA.

Architectural Comparison: PROM vs. PLA vs. PAL



3.2 Complex Programmable Logic Devices (CPLD)

A CPLD integrates multiple PAL/GAL-like **Function Blocks (Logic Array Blocks)** interconnected through a centralized, uniform **Global Interconnect Matrix (GIM)**.

- Features non-volatile EEPROM/Flash configuration storage (starts up instantly upon power application).
- Delivers predictable, deterministic pin-to-pin propagation timing delays.
- Typically contains tens to hundreds of macrocells (equivalent to 1,000 to 100,000 logic gates).

3.3 Field Programmable Gate Arrays (FPGA)

FPGAs are high-density digital ICs based on an **island-style architecture** consisting of a two-dimensional matrix of programmable logic resources:

- **Configurable Logic Blocks (CLBs):** The fundamental computational building blocks. Each CLB contains:
  - **Look-Up Tables (LUTs):** A k-input LUT is a small high-speed SRAM block ($2^k \times 1$) that implements **any arbitrary Boolean logic function of k variables** simply by loading the function's truth table truth bits into memory. The inputs serve as address lines selecting the precomputed output bit.
  - **Storage Elements:** Configurable D flip-flops / latches with clock enables, set/reset lines.
  - **Arithmetic Carry Logic:** Dedicated ultra-fast lookahead carry chains for high-speed addition.

© Biraj

  - **Multiplexers:** Wide multiplexers for combining multiple LUT outputs.

- **Programmable Interconnects:** Segmented horizontal and vertical routing channels joined at programmable **Switch Boxes** and **Connection Boxes**.
- **Input/Output Blocks (IOBs):** Configurable bidirectional input/output buffers surrounding the periphery, supporting multiple I/O standards (LVTTL, LVCMOS, differential signaling).
- **Specialized Hardware Blocks:** Modern FPGAs embed dedicated DSP slices (hardwired MAC units), dual-port block RAMs (BRAM), PLLs/DCMs, and high-speed transceivers.

Feature	CPLD	FPGA
Architecture	Coarse-grained (PAL-like function blocks)	Fine-grained (2D array of CLBs and LUTs)
Configuration Memory	Non-volatile (Internal Flash/EEPROM)	Volatile (SRAM-based, requires boot ROM)
Boot-up Time	Instant-on upon power application	Requires configuration bitstream loading
Interconnect	Global interconnect matrix (Centralized)	Segmented channels and switch matrices
Timing Predictability	Deterministic, fixed pin-to-pin delay	Variable, placement and routing dependent
Density / Logic Capacity	Low to moderate ($< 10{,}000$ macrocells)	Extremely High (Millions of system gates)
Typical Applications	Glue logic, boot controllers, address decoders	DSP, telecommunications, AI acceleration, ASIC prototyping

4. Digital-to-Analog Converters (DAC)

A DAC translates an n-bit binary digital word $D = b_1 2^{-1} + b_2 2^{-2} + \cdots + b_n 2^{-n}$ into a proportional analog voltage or current.

4.1 DAC Specifications

- **Resolution:** The number of input bits (n), or the smallest output voltage increment (step size):

$$\text{Step Size} = \frac{V_{FS}}{2^n - 1} \approx \frac{V_{\text{ref}}}{2^n} \tag{39}$$

- **Settling Time:** The time elapsed from an input code transition until the analog output settles to within $\pm\frac{1}{2}$ LSB of its final value.
- **Linearity Errors:** Integral Non-Linearity (INL) and Differential Non-Linearity (DNL).

© Biraj

- **Monotonicity:** A DAC is monotonic if its output voltage never decreases when the input digital code increases.

4.2 Binary-Weighted Resistor DAC

Uses an inverting operational amplifier summing configuration with binary-weighted input resistors:

$$R_1 = R, \quad R_2 = 2R, \quad R_3 = 4R, \quad \dots, \quad R_n = 2^{n-1}R \tag{40}$$

The output analog voltage is given by:

$$V_o = -I_T R_f = -V_{\text{ref}} \frac{R_f}{R} \left(\frac{b_1}{2^1} + \frac{b_2}{2^2} + \frac{b_3}{2^4} + \cdots + \frac{b_n}{2^n} \right) \tag{41}$$

Major Drawbacks:

1. **Wide Range of Resistance Values:** For an 8-bit DAC, the resistor ratio is $2^7 : 1 = 128 : 1$; for a 16-bit DAC, it reaches $32{,}768 : 1$.
2. Extremely difficult to fabricate widely disparate, tightly matched resistors with identical temperature coefficients on an integrated circuit die.
3. The MSB resistor (R) draws significantly higher current than the LSB resistor, causing uneven thermal dissipation.

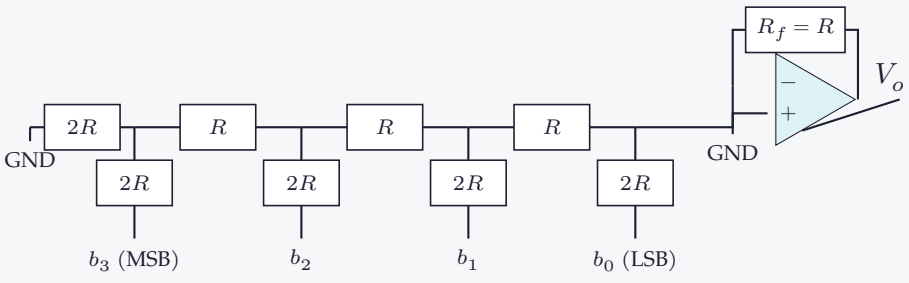
4.3 R-$2R$ Ladder DAC

The R-$2R$ ladder DAC overcomes all drawbacks of the binary-weighted design by requiring **only two distinct resistor values** (R **and** $2R$) regardless of resolution n.

Thevenin Resistance Principle of the R-$2R$ Ladder

At every ladder node, the equivalent resistance looking toward the terminating end or down any branch is always exactly R. At each junction, the incoming current splits evenly in half (50% down the branch through $2R$, and 50% onward through R).

4-Bit R-$2R$ Ladder DAC Circuit Architecture



For an n-bit R-$2R$ ladder DAC, the analog output voltage is:

$$V_o = -V_{\text{ref}} \frac{R_f}{R} \sum_{i=1}^{n} \frac{b_i}{2^i} = -V_{\text{ref}} \left(\frac{b_1}{2} + \frac{b_2}{4} + \cdots + \frac{b_n}{2^n} \right) \quad (\text{when } R_f = R) \tag{42}$$

Key Advantages:

- Requires only two values: R and $2R$.

© Biraj

- Highly practical for monolithic semiconductor IC fabrication using thin-film laser-trimmed resistors.
- Easily scalable to high bit-resolutions (12, 16, or 24 bits).

5. Analog-to-Digital Converters (ADC)

An ADC converts a continuous-time, continuous-amplitude analog voltage V_{in} into a discrete n-bit digital code via three sequential steps: **Sampling, Quantization, and Binary Encoding**.

5.1 Quantization Error and ADC Dynamic Range

The ideal quantization step size is $\Delta = \frac{V_{FS}}{2^n - 1}$. The maximum inherent quantization uncertainty is:

$$Q_e = \pm \frac{\Delta}{2} \tag{43}$$

The theoretical Signal-to-Quantization-Noise Ratio (SQNR) for a full-scale sinusoidal input is:

$$\text{SQNR} = 6.02n + 1.76 \text{ dB} \tag{44}$$

Every added bit of resolution improves dynamic range by approximately 6 dB.

5.2 Flash (Simultaneous / Parallel) ADC

The Flash ADC is the **fastest possible ADC architecture**.

- **Architecture:** Contains a precision reference voltage divider of 2^n identical resistors generating $2^n - 1$ reference thresholds, $2^n - 1$ analog comparators operating in parallel, and a priority encoder.
- **Operation:** All $2^n - 1$ comparators simultaneously compare V_{in} against their assigned reference levels in a single clock cycle, generating a thermometer code which is converted to binary by the priority encoder.
- **Conversion Time:** Strictly 1 clock cycle ($O(1)$ time complexity, typically $<$ 1–5 ns).
- **Limitation:** Circuit hardware grows exponentially ($2^n - 1$ comparators). An 8-bit Flash ADC requires 255 comparators; a 10-bit converter requires 1,023 comparators, resulting in huge silicon area and excessive power consumption.

5.3 Successive Approximation Register (SAR) ADC

The SAR ADC utilizes a **binary search algorithm** to systematically approximate the input voltage bit by bit from MSB down to LSB using a single comparator and an internal DAC.

1. **Cycle 1:** The SAR sets the MSB ($b_1 = 1$, all others 0). The DAC generates $V_{\text{DAC}} = \frac{1}{2} V_{\text{ref}}$.
2. If $V_{in} > V_{\text{DAC}}$, the comparator output is HIGH; the MSB is retained ($b_1 = 1$). If $V_{in} < V_{\text{DAC}}$, the MSB is cleared ($b_1 = 0$).
3. **Cycle 2 to n:** The process is repeated sequentially for each successive bit ($b_2, b_3, \ldots, b_n$).

© Biraj

4. **Conversion Time:** Strictly constant and directly proportional to resolution:

$$T_C = n \times T_{\text{clk}} \tag{45}$$

5. **Applications:** The industry standard for medium-to-high speed (100 kSPS to 10 MSPS), 10–18 bit instrumentation, biomedical data acquisition (ECG/EEG), and microcontroller embedded ADCs.

5.4 Dual-Slope Integrating ADC

The Dual-Slope ADC provides the **highest accuracy and superior noise rejection** through continuous analog integration.

Dual-Slope Conversion Derivation

- **Phase 1 (Fixed-Time Run-Up):** Switch connects to V_{in} for a fixed time $T_1 = 2^n \times T_{\text{clk}}$. The integrator output ramps to a peak voltage:

$$V_{\text{peak}} = -\frac{1}{RC}\int_0^{T_1} V_{in}\, dt = -\frac{V_{in}T_1}{RC} \tag{46}$$

- **Phase 2 (Variable-Time Run-Down):** Switch connects to a negative reference $-V_{\text{ref}}$. The integrator ramps back toward zero at a constant slope:

$$V_o(t) = V_{\text{peak}} + \frac{V_{\text{ref}}t}{RC} \tag{47}$$

  When $V_o(t) = 0$ at time $t = T_2$, the comparator stops the digital counter.

- Equating integrator charges:

$$\frac{V_{in}T_1}{RC} = \frac{V_{\text{ref}}T_2}{RC} \implies T_2 = T_1\left(\frac{V_{in}}{V_{\text{ref}}}\right) \tag{48}$$

Outstanding Advantages:

- **Immune to Component Drift:** Conversion time T_2 is **entirely independent of the integration resistance R, capacitance C, and clock frequency f_{clk}** because they cancel out completely in the ratio.
- **Power-Line Noise Rejection:** By setting integration period T_1 to an exact integer multiple of the 50 Hz or 60 Hz power-line cycle (20 ms or 16.67 ms), hum and line noise integrate to identically zero.
- **Limitation:** Very slow conversion ($T_{C(\text{max})} = 2 \times 2^n \times T_{\text{clk}}$). Standard choice for precision Digital Multimeters (DMMs) and digital weigh scales.

© Biraj

Parameter	Flash ADC	SAR ADC	Dual-Slope ADC	Counter / Ramp ADC
Conversion Speed	Extremely Fast (1–5 ns)	Fast (1–10 μs)	Extremely Slow (10–100 ms)	Slow (Variable)
Clock Cycles	1 clock cycle ($O(1)$)	n clock cycles	Up to 2×2^n cycles	Up to 2^n cycles
Resolution	6–8 bits	8–18 bits	16–24 bits (Highest)	8–10 bits

Parameter (contd.)	Flash ADC	SAR ADC	Dual-Slope ADC	Counter / Ramp ADC
Hardware Cost	High ($2^n - 1$ comparators)	Moderate (1 comp, 1 DAC)	Low (1 integrator, 1 comp)	Low (1 counter, 1 DAC)
Noise Immunity	Poor	Moderate	Exceptional (50/60 Hz hum)	Poor
Typical Uses	Radar, oscilloscopes, video	Microcontrollers, DSP	DMMs, weighing scales	Low-cost sensors

6. Sample and Hold (S/H) Circuit

When converting high-frequency analog signals, any input variation during the finite conversion interval (t_c) introduces **aperture error**. A Sample-and-Hold circuit freezes the analog voltage for the duration of the ADC conversion.

6.1 Circuit Topology and Modes of Operation

A basic S/H circuit consists of an input unity-gain buffer op-amp, a high-speed analog switch (JFET or MOSFET), an ultra-low leakage **Hold Capacitor** (C_H), and an output high-input-impedance unity-gain buffer op-amp.

- **Sample (Track) Mode:** The switch is closed (ON). The hold capacitor C_H charges to follow the input analog voltage V_{in} through the switch ON resistance R_{on}.
- **Hold Mode:** The switch is opened (OFF). The charge trapped on C_H maintains the instantaneous voltage constant. The high-impedance buffer delivers this stable voltage to the ADC.

6.2 Key Dynamic Performance Parameters

- **Acquisition Time (t_{ac}):** Time required for the hold capacitor voltage to charge and settle to within a specified error band (e.g., 0.01% or $\pm\frac{1}{2}$ LSB) following a transition from hold to sample mode. Governed by switch resistance and capacitance:

$$t_{ac} \approx k \cdot R_{\text{on}} \cdot C_H \quad \text{(where } k \approx 7\text{–}9 \text{ for 12-bit accuracy)} \tag{49}$$

© Biraj

- **Aperture Time (t_{ap}):** The finite delay between the initiation of the hold command and the effective physical opening of the sampling switch.
- **Aperture Jitter (t_j):** The sample-to-sample statistical timing uncertainty in switch opening. The maximum input signal frequency before aperture jitter exceeds 1 LSB is:

$$f_{\text{max}} = \frac{1}{2\pi \cdot t_j \cdot 2^n} \tag{50}$$

- **Droop Rate ($\frac{dV}{dt}$):** The rate at which the output voltage drifts during hold mode due to capacitor discharge through parasitic leakage currents:

$$\text{Droop Rate} = \frac{I_{\text{leak}}}{C_H} \tag{51}$$

- **Trade-off in Sizing C_H:** A large C_H reduces droop rate but increases acquisition time; a small C_H accelerates acquisition but degrades droop performance.

© Biraj

7. Quick Revision Page

Key Formulas and Characteristic Relations at a Glance

- **Digital IC Speed-Power Metric:**

$$t_{pd} = \frac{t_{PLH} + t_{PHL}}{2}, \quad \text{PDP} = P_D \times t_{pd}, \quad P_{\text{dyn(CMOS)}} = \alpha C_L V_{DD}^2 f$$

- **Noise Margins and Open-Collector Pull-up Resistor:**

$$NM_H = V_{OH(\text{min})} - V_{IH(\text{min})}, \quad NM_L = V_{IL(\text{max})} - V_{OL(\text{max})}, \quad R_{P(\text{min})} = \frac{V_{CC} - V_{OL}}{I_{OL} - N \cdot I_{IL}}$$

- **DAC Resolution and Analog Output (R-$2R$ Ladder):**

$$\text{Step Size} = \frac{V_{FS}}{2^n - 1}, \quad V_o = -V_{\text{ref}} \left(\frac{b_1}{2} + \frac{b_2}{4} + \cdots + \frac{b_n}{2^n} \right)$$

- **ADC Timing and Dual-Slope Relation:**

$$T_{C(\text{SAR})} = n \cdot T_{\text{clk}}, \quad T_{C(\text{Flash})} = 1 \cdot T_{\text{clk}}, \quad T_2 = T_1 \cdot \left(\frac{V_{in}}{V_{\text{ref}}} \right)$$

One-Line High-Yield Definitions

- **Noise Margin:** The maximum noise voltage amplitude that can be added to an input signal without corrupting the recognized logic level.
- **Totem-Pole Output:** An active BJT push-pull output driver providing low impedance during both pull-up and pull-down transitions.
- **ECL Logic Family:** Non-saturating BJT current-steering logic that avoids saturation charge storage to achieve sub-nanosecond speeds.
- **Static RAM (SRAM):** Volatile memory storing data in bi-stable cross-coupled CMOS inverters requiring no periodic refresh.
- **Dynamic RAM (DRAM):** Volatile memory storing data as charge on a 1T-1C capacitor requiring periodic refresh cycles.
- **Look-Up Table (LUT):** An SRAM-based truth table in an FPGA that realizes any arbitrary k-variable logic function.
- **Flash ADC:** A parallel ADC utilizing $2^n - 1$ simultaneous comparators to perform conversions in a single clock cycle.
- **Dual-Slope ADC:** An integrating ADC whose conversion is independent of R, C, and clock frequency, offering high noise rejection.
- **SAR ADC:** An analog-to-digital converter that determines binary bits sequentially via a binary search algorithm in n cycles.

© Biraj

- **Droop Rate:** The rate of voltage decay across the hold capacitor in a Sample-and-Hold circuit caused by parasitic leakage.

Frequently Asked / Viva Questions

1. **Why can totem-pole TTL outputs not be connected in a wired-AND configuration?**

   If one gate asserts HIGH and the other LOW, a direct low-resistance short circuit from V_{CC} to ground through Q_3 and Q_4 occurs, producing damaging current spikes and destruction.

2. **Why is ECL significantly faster than TTL?**

   ECL operates BJTs strictly in the non-saturating active region using a differential current-steering switch. Eliminating saturation eliminates base minority carrier storage time ($t_s = 0$).

3. **Why does static CMOS consume virtually zero power at DC?**

   For any stable logic input, either the PMOS pull-up network or the NMOS pull-down network is completely turned OFF, cutting off any direct path between V_{DD} and ground.

4. **Why must PMOS transistors be made wider than NMOS in CMOS inverters?**

   Because electron mobility in silicon is approximately 2.5 times higher than hole mobility ($\mu_n \approx 2.5\mu_p$). PMOS width must be scaled by $2.5\times$ to equalize drive currents.

5. **Why does DRAM require periodic refreshing while SRAM does not?**

   DRAM stores data as charge on tiny capacitors ($C_s \approx 30$ fF) which leak through semiconductor junctions within milliseconds. SRAM uses cross-coupled active latching inverters.

6. **Explain the functional architectural difference between PLA and PAL.**

   A PLA features both programmable AND and programmable OR arrays, offering maximum term sharing. A PAL has a programmable AND array and a fixed OR array, offering faster speed and simpler structure.

7. **What is a Look-Up Table (LUT) in an FPGA?**

   An LUT is a small SRAM block storing the truth table values of a logic function. For a 4-input LUT, 4 input lines act as addresses selecting 1 of 16 stored output bits.

8. **Why is an R-$2R$ ladder DAC preferred over a binary-weighted resistor DAC?**

   An R-$2R$ ladder requires only two resistor values (R and $2R$) regardless of bit resolution, avoiding the fabrication of massive, tightly matched resistor ranges.

9. **How many comparators are required for an 8-bit Flash ADC?**

   An 8-bit Flash ADC requires $2^8 - 1 = 255$ parallel comparators.

10. **Why is conversion time constant in a SAR ADC regardless of input voltage?**

    Because a SAR ADC always executes a fixed n-step binary search sequence, testing one bit per clock cycle from MSB to LSB.

© Biraj

Frequently Asked / Viva Questions (contd.)

11. **Why is the Dual-Slope ADC immune to variations in resistance and capacitance?**

    Because the integrator component values R and C appear identically in both the run-up and run-down equations and cancel out completely in the ratio $T_2 = T_1(V_{in}/V_{\text{ref}})$.

12. **What causes aperture jitter in a Sample-and-Hold circuit, and why is it problematic?**

    Aperture jitter is the sample-to-sample statistical variation in the switch opening moment, which introduces phase noise and voltage errors when sampling fast-changing analog signals.

Summary Matrix: Comparison of ADC Architectures

Metric	Flash ADC	SAR ADC	Dual-Slope ADC
Conversion Time (T_C)	1 clock cycle ($O(1)$)	n clock cycles ($O(n)$)	Up to 2^{n+1} cycles ($O(2^n)$)
Relative Speed	Ultra-fast (>1 GSPS)	Medium-high (100 kSPS–10 MSPS)	Very slow (10–100 SPS)
Hardware Complexity	Exponential ($2^n - 1$ comparators)	Moderate (1 comp, 1 DAC, SAR)	Minimal (1 integrator, 1 comp)
Resolution Limits	6–8 bits (rarely 10)	8–18 bits	16–24 bits (Extreme precision)
Noise Rejection	Poor	Moderate	Superb (line hum canceling)
Dominant Usage	Oscilloscopes, SDR, radar	Embedded MCUs, audio, DSP	Digital multimeters, scales

© Biraj

Module 4

The VLSI Design Flow and Methodologies

1. The VLSI Design Flow and Methodologies

Very Large Scale Integration (VLSI) design is the systematic transformation of an abstract architectural system specification into a physically verified, fabricated silicon integrated circuit containing millions to billions of transistors.

1.1 The Gajski-Kuhn Y-Chart

The Gajski-Kuhn Y-chart visualizes VLSI design complexity across three distinct **domains of representation** intersecting at concentric **levels of abstraction**:

- **Behavioral Domain (What it does):** Specifies functional algorithms, equations, input-output relationships, and state transitions without detailing physical hardware.
- **Structural Domain (How it is composed):** Defines hardware entities as interconnections of structural sub-blocks, modules, logic gates, and transistors.
- **Physical / Geometric Domain (How it is built):** Represents physical implementation as silicon geometric masks, transistor layouts, placement coordinates, and metallization routing tracks.
- **Levels of Abstraction:** Architectural / System level $\rightarrow$ Algorithmic level $\rightarrow$ Register-Transfer Level (RTL) $\rightarrow$ Logic / Gate level $\rightarrow$ Transistor / Circuit level $\rightarrow$ Mask layout level.

1.2 Step-by-Step Modern Digital ASIC and FPGA Design Flow

1. **System Specification & Architecture Planning:** Defines system throughput, clock frequency, word width, power budget, and interface protocols.
2. **Design Entry:** Entering the digital design into CAD tools:
    - *Schematic Capture:* Graphical drawing of interconnected logic gates. Intuitive for small designs; unmanageable for complex million-gate chips.
    - *State Diagram / FSM Entry:* Graphical specification of states, transitions, and outputs; automatically compiled into HDL code.
    - *Hardware Description Languages (HDLs):* Textual algorithmic descriptions using VHDL, Verilog, or SystemVerilog. Standard for modern complex VLSI.
3. **Functional (RTL) Simulation:** Verifies behavioral and logic correctness using an HDL testbench. Assumes ideal zero-delay or unit-delay gates to detect algorithmic bugs early.

© Biraj

4. **Logic Synthesis:** CAD tool (e.g., Synopsys Design Compiler, Vivado) translates abstract RTL code into an optimized, technology-specific **gate-level netlist** mapped to standard cells (AND, NAND, Flip-Flops) from a target semiconductor library, driven by designer constraints (target clock period, maximum silicon area, dynamic power).

5. **Gate-Level Netlist Simulation:** Simulates the synthesized gate netlist to confirm that synthesis introduced no unintended logic changes or state mismatches.

6. **Static Timing Analysis (STA):** Exhaustively evaluates path delays to guarantee that clock setup and hold time requirements are met for every register without dynamic vectors:

$$\text{Setup Time Constraint:} \quad T_{\text{clk}} \geq t_{cq} + t_{\text{comb(max)}} + t_{\text{setup}} - t_{\text{skew}} \tag{52}$$

$$\text{Hold Time Constraint:} \quad t_{cq} + t_{\text{comb(min)}} \geq t_{\text{hold}} + t_{\text{skew}} \tag{53}$$

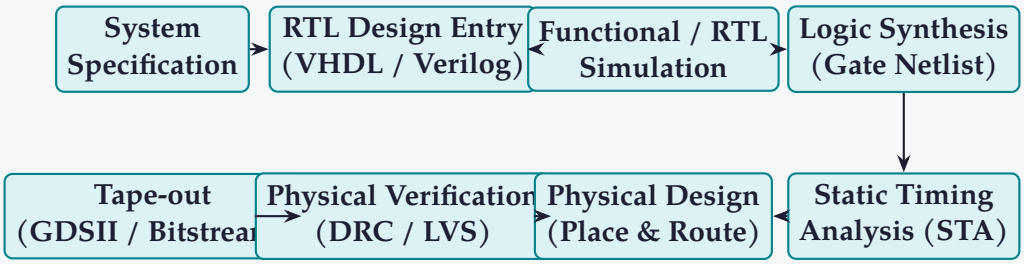
7. **Physical Design (P&R - Place and Route):**
   - *Floorplanning:* Allocates chip area for major functional blocks, I/O pads, and power/ground grids (V_{DD}, V_{SS}).
   - *Placement:* Determines exact (x, y) coordinate locations for every standard cell.
   - *Clock Tree Synthesis (CTS):* Constructs balanced clock distribution networks (H-tree) to minimize clock skew (t_{skew}) and insertion delay.
   - *Routing:* Wires together all cell pins using multi-layer metallization.

8. **Parasitic Extraction and Post-Layout Timing Verification:** Extracts parasitic resistances (R) and capacitances (C) from actual layout geometries and back-annotates them into a Standard Delay Format (SDF) file for final timing sign-off.

9. **Physical Verification:**
   - *DRC (Design Rule Checking):* Verifies that geometric shapes satisfy foundry manufacturing rules (minimum metal width, spacing, enclosure).
   - *LVS (Layout Versus Schematic):* Extracts electrical transistor circuits from the physical layout and compares connectivity netlists 1-to-1 against the schematic.

10. **Tape-out:** Exporting final layout data in GDSII / OASIS stream format to the semiconductor foundry for photolithographic mask fabrication (or generating a configuration bitstream for an FPGA).

Digital VLSI / ASIC Design Flow Pipeline



© Biraj

2. VHDL Language Fundamentals and Structural Units

VHDL (VHSIC Hardware Description Language) is an IEEE standard (IEEE Std 1076) language designed to model the behavior, dataflow, and structural composition of digital electronic systems across multiple levels of abstraction.

2.1 Fundamental Design Units of VHDL

A complete VHDL design entity is composed of primary design units:

1. **Library and Package Declarations:** Grants access to pre-compiled packages containing standardized data types, constants, and operators:

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;     -- Imports STD_LOGIC, STD_LOGIC_VECTOR
USE IEEE.NUMERIC_STD.ALL;        -- Imports SIGNED, UNSIGNED arithmetic
```

2. **Entity Declaration:** Defines the external hardware boundary and interface ports of the module (the "black box" view):

```
ENTITY entity_name IS
 GENERIC ( param_name : type := default_val ); -- Optional parameterized constants
  PORT (
    port_name : mode data_type;
    ...
  );
END entity_name;
```

3. **Architecture Body:** Defines the internal behavior, logic equations, or structural interconnection of components (the "inside" of the box). Multiple architectures can be written for a single entity (e.g., behavioral vs. structural):

```
ARCHITECTURE arch_name OF entity_name IS
  -- Declarative region: signals, constants, components, types
BEGIN
  -- Concurrent statements region: processes, component instantiations
END arch_name;
```

4. **Configuration Declaration:** Binds specific architecture bodies and component implementations to entity instances in hierarchical designs.

© Biraj

2.2 VHDL Port Modes

Port Mode	Directionality	Internal Architecture Usage
`IN`	**Input to module**	Read-only inside architecture. Cannot be assigned or driven.
`OUT`	**Output from module**	Write-only. Values can be assigned to it, but cannot be read internally.
`INOUT`	**Bidirectional**	Can be driven and read internally. Requires tri-state resolution.
`BUFFER`	**Output with internal feedback**	Drives output pin and allows internal readback. Has restrictive binding rules.

3. Data Types, Data Objects, and Operators

3.1 Data Objects in VHDL

VHDL supports four classes of data objects:

- **SIGNAL:** Represents physical interconnecting electrical wires or traces.
  - Declared in architecture declarative regions or packages; visible throughout the entire architecture.
  - Updated concurrently with a scheduled propagation delay (delta delay δ).
  - Assignment operator: `<=` (e.g., `Y <= A AND B;`).
- **VARIABLE:** Represents local computational scratchpad storage.
  - Declared strictly inside a `PROCESS`, `FUNCTION`, or `PROCEDURE`. Not visible outside.
  - Updated **instantaneously** without delay.
  - Assignment operator: `:=` (e.g., `v_count := v_count + 1;`).
- **CONSTANT:** Holds a permanent, unmodifiable value established during elaboration. Assignment operator: `:=` (e.g., `CONSTANT WIDTH : INTEGER := 8;`).
- **FILE:** Represents external host system files used for automated testbench verification and stimulus vectors.

© Biraj

Property	SIGNAL	VARIABLE
Hardware Meaning	Physical electrical wire / bus	Local memory / register variable
Declaration Scope	Architecture, package, block	Exclusively inside `PROCESS` or subprograms
Assignment Operator	`<=` (Non-blocking, scheduled)	`:=` (Blocking, instantaneous)
Update Timing	Updated at end of current simulation cycle	Updated immediately when evaluated
Synthesized Hardware	Interconnecting wires, latches, or flip-flops	Internal combinational nodes or registers

3.2 Standard and IEEE 1164 Data Types

- **Standard Types:** `BIT` (`'0'`, `'1'`), `BOOLEAN` (`TRUE`, `FALSE`), `INTEGER` (32-bit minimum range: -2^{31} to $2^{31}-1$), `CHARACTER`, `TIME` (e.g., `10 ns`).
- **IEEE Standard 1164 Multi-Valued Logic (`STD_LOGIC`):** Provides realistic modeling of electrical logic states across integrated circuit nodes:

Value	Electrical Meaning	Hardware Significance
`'U'`	**Uninitialized**	Default initial simulation state of unassigned signals
`'X'`	**Forcing Unknown**	Unresolvable contention / unknown logic conflict
`'0'`	**Forcing 0**	Strong logic LOW tied to ground (V_{SS})
`'1'`	**Forcing 1**	Strong logic HIGH tied to supply (V_{DD})
`'Z'`	**High Impedance**	Tri-state output; electrically disconnected
`'W'`	**Weak Unknown**	Weak signal contention
`'L'`	**Weak 0**	Pull-down resistor to ground
`'H'`	**Weak 1**	Pull-up resistor to supply
`'-'`	**Don't Care**	Wildcard used in synthesis minimization

3.3 VHDL Operators

- **Logical Operators:** `AND`, `OR`, `NAND`, `NOR`, `XOR`, `XNOR`, `NOT`.
- **Relational Operators:** `=` (equal), `/=` (not equal), `<`, `<=`, `>`, `>=`.
- **Arithmetic Operators:** `+` (addition), `-` (subtraction), `*` (multiplication), `/` (division), `MOD`, `REM`, `ABS`, `**` (exponentiation).

© Biraj

- **Concatenation Operator (`&`):** Combines scalars or vectors into wider vectors (e.g., `Y <= A & B;`).

4. VHDL Modeling Styles and Coding Methodologies

Digital hardware architectures can be expressed using three distinct modeling styles:

4.1 Dataflow (Concurrent) Modeling

Expresses hardware as continuous concurrent signal assignments that execute simultaneously whenever any right-hand-side operand changes:

- **Simple Concurrent Assignment:** `Y <= A AND B;`
- **Conditional Signal Assignment (`WHEN-ELSE`):**

```
Y <= A WHEN (Sel = "00") ELSE
     B WHEN (Sel = "01") ELSE
     C WHEN (Sel = "10") ELSE D;
```

- **Selected Signal Assignment (`WITH-SELECT`):**

```
WITH Sel SELECT
  Y <= A WHEN "00",
       B WHEN "01",
       C WHEN "10",
       D WHEN OTHERS;
```

4.2 Behavioral (Sequential) Modeling

Specifies the algorithmic behavior of the hardware within a **`PROCESS`** block. Statements inside a process execute sequentially from top to bottom, but the process block itself is evaluated concurrently relative to the rest of the architecture:

- **Sensitivity List:** A list of signals enclosed in parentheses following the `PROCESS` keyword. The process activates whenever any signal in its sensitivity list transitions:

```
PROCESS (clk, reset) -- Activated on transitions of clk or reset
BEGIN
  IF (reset = '1') THEN
    Q <= '0';
  ELSIF rising_edge(clk) THEN
    Q <= D;
  END IF;
END PROCESS;
```

- **Sequential Control Statements:** `IF-THEN-ELSIF-ELSE`, `CASE-WHEN`, `FOR-LOOP`, `WHILE-LOOP`.

© Biraj

Critical Synthesis Hazard: Inferred Latches

In combinational processes, if an `IF` statement lacks a complete `ELSE` clause, or a `CASE` statement omits the `WHEN OTHERS` branch or fails to assign every output signal across all possible input permutations, the synthesis tool must infer a **transparent latch** to preserve the previous output state. This creates asynchronous hazards, degrades timing closure, and must be prevented by assigning unambiguous default values at the top of every combinational process.

4.3 Structural Modeling

Describes a system hierarchically by declaring pre-designed sub-components and mapping their ports together like an electrical schematic:

1. Declare component interface in architecture declarative region.
2. Instantiate components in architecture body and connect signals using `PORT MAP`.

5. VHDL Code Library for Combinational Logic Circuits

1. Half Adder and Full Adder (Dataflow, Behavioral, Structural)

```
-- Half Adder (Dataflow)
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
ENTITY half_adder IS
  PORT ( A, B : IN STD_LOGIC; Sum, Cout : OUT STD_LOGIC );
END half_adder;
ARCHITECTURE dataflow OF half_adder IS
BEGIN
  Sum  <= A XOR B;
  Cout <= A AND B;
END dataflow;

-- Full Adder (Structural using 2 Half Adders + OR Gate)
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
ENTITY full_adder IS
  PORT ( A, B, Cin : IN STD_LOGIC; Sum, Cout : OUT STD_LOGIC );
END full_adder;
ARCHITECTURE structural OF full_adder IS
  COMPONENT half_adder
    PORT ( A, B : IN STD_LOGIC; Sum, Cout : OUT STD_LOGIC );
  END COMPONENT;
  SIGNAL s1, c1, c2 : STD_LOGIC;
BEGIN
  HA1: half_adder PORT MAP ( A => A, B => B, Sum => s1, Cout => c1 );
  HA2: half_adder PORT MAP ( A => s1, B => Cin, Sum => Sum, Cout => c2 );
  Cout <= c1 OR c2;
```


© Biraj

```
END structural;
```

2. 4-to-1 Multiplexer (Dataflow and Behavioral)

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;

ENTITY mux4to1 IS
  PORT (
    D   : IN  STD_LOGIC_VECTOR(3 DOWNTO 0);
    Sel : IN  STD_LOGIC_VECTOR(1 DOWNTO 0);
    Y   : OUT STD_LOGIC
  );
END mux4to1;

ARCHITECTURE behavioral OF mux4to1 IS
BEGIN
  PROCESS (D, Sel)
  BEGIN
    CASE Sel IS
      WHEN "00" => Y <= D(0);
      WHEN "01" => Y <= D(1);
      WHEN "10" => Y <= D(2);
      WHEN "11" => Y <= D(3);
      WHEN OTHERS => Y <= '0';
    END CASE;
  END PROCESS;
END behavioral;
```

3. 2-to-4 Binary Decoder with Active-Low Enable

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;

ENTITY decoder2to4 IS
  PORT (
    A     : IN  STD_LOGIC_VECTOR(1 DOWNTO 0);
    En_n  : IN  STD_LOGIC;                      -- Active-low enable
    Y     : OUT STD_LOGIC_VECTOR(3 DOWNTO 0)    -- Active-high outputs
  );
END decoder2to4;

ARCHITECTURE dataflow OF decoder2to4 IS
BEGIN
  Y <= "0001" WHEN (En_n = '0' AND A = "00") ELSE
       "0010" WHEN (En_n = '0' AND A = "01") ELSE
       "0100" WHEN (En_n = '0' AND A = "10") ELSE
       "1000" WHEN (En_n = '0' AND A = "11") ELSE
```


© Biraj

```
        "0000";
END dataflow;
```

4. 4-Bit Arithmetic Logic Unit (ALU)

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.NUMERIC_STD.ALL;

ENTITY alu_4bit IS
  PORT (
    A, B   : IN  STD_LOGIC_VECTOR(3 DOWNTO 0);
    Op     : IN  STD_LOGIC_VECTOR(2 DOWNTO 0);
    Result : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
    Cout   : OUT STD_LOGIC
  );
END alu_4bit;

ARCHITECTURE behavioral OF alu_4bit IS
BEGIN
  PROCESS (A, B, Op)
    VARIABLE temp : UNSIGNED(4 DOWNTO 0);
  BEGIN
    Cout <= '0';
    CASE Op IS
      WHEN "000" => -- ADD
        temp := ('0' & UNSIGNED(A)) + ('0' & UNSIGNED(B));
        Result <= STD_LOGIC_VECTOR(temp(3 DOWNTO 0));
        Cout   <= temp(4);
      WHEN "001" => -- SUBTRACT
        temp := ('0' & UNSIGNED(A)) - ('0' & UNSIGNED(B));
        Result <= STD_LOGIC_VECTOR(temp(3 DOWNTO 0));
      WHEN "010" => -- AND
        Result <= A AND B;
      WHEN "011" => -- OR
        Result <= A OR B;
      WHEN "100" => -- XOR
        Result <= A XOR B;
      WHEN "101" => -- NOT A
        Result <= NOT A;
      WHEN OTHERS =>
        Result <= "0000";
    END CASE;
  END PROCESS;
END behavioral;
```


© Biraj

6. VHDL Code Library for Sequential Logic Circuits

1. D Flip-Flop with Asynchronous Active-Low Reset

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;

ENTITY d_flip_flop IS
  PORT (
    clk     : IN  STD_LOGIC;
    rst_n   : IN  STD_LOGIC; -- Asynchronous active-low reset
    d       : IN  STD_LOGIC;
    q, q_bar: OUT STD_LOGIC
  );
END d_flip_flop;

ARCHITECTURE behavioral OF d_flip_flop IS
  SIGNAL q_reg : STD_LOGIC;
BEGIN
  PROCESS (clk, rst_n)
  BEGIN
    IF (rst_n = '0') THEN
      q_reg <= '0';
    ELSIF rising_edge(clk) THEN
      q_reg <= d;
    END IF;
  END PROCESS;
  q     <= q_reg;
  q_bar <= NOT q_reg;
END behavioral;
```

2. 4-Bit Synchronous Up-Counter with Parallel Load

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.NUMERIC_STD.ALL;

ENTITY counter_4bit IS
  PORT (
    clk    : IN  STD_LOGIC;
   rst_n  : IN  STD_LOGIC;                -- Asynchronous active-low reset
    load   : IN  STD_LOGIC;                    -- Synchronous load enable
    en     : IN  STD_LOGIC;                      -- Count enable
    d_in   : IN  STD_LOGIC_VECTOR(3 DOWNTO 0);
    q_out  : OUT STD_LOGIC_VECTOR(3 DOWNTO 0)
  );
END counter_4bit;
```


© Biraj

```
ARCHITECTURE behavioral OF counter_4bit IS
  SIGNAL count : UNSIGNED(3 DOWNTO 0);
BEGIN
  PROCESS (clk, rst_n)
  BEGIN
    IF (rst_n = '0') THEN
      count <= (OTHERS => '0');
    ELSIF rising_edge(clk) THEN
      IF (load = '1') THEN
        count <= UNSIGNED(d_in);
      ELSIF (en = '1') THEN
        count <= count + 1;
      END IF;
    END IF;
  END PROCESS;
  q_out <= STD_LOGIC_VECTOR(count);
END behavioral;
```

3. 4-Bit Universal Shift Register

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;

ENTITY universal_shift_reg IS
  PORT (
    clk    : IN  STD_LOGIC;
    rst_n  : IN  STD_LOGIC;
   mode   : IN  STD_LOGIC_VECTOR(1 DOWNTO 0); -- 00: Hold, 01: Right, 10: Left, 11: Load
    sin_r  : IN  STD_LOGIC;                  -- Serial input for shift-right
    sin_l  : IN  STD_LOGIC;                   -- Serial input for shift-left
    pin    : IN  STD_LOGIC_VECTOR(3 DOWNTO 0);  -- Parallel data in
    pout   : OUT STD_LOGIC_VECTOR(3 DOWNTO 0)   -- Parallel data out
  );
END universal_shift_reg;

ARCHITECTURE behavioral OF universal_shift_reg IS
  SIGNAL r_reg : STD_LOGIC_VECTOR(3 DOWNTO 0);
BEGIN
  PROCESS (clk, rst_n)
  BEGIN
    IF (rst_n = '0') THEN
      r_reg <= (OTHERS => '0');
    ELSIF rising_edge(clk) THEN
      CASE mode IS
        WHEN "00" => r_reg <= r_reg;                    -- Hold
        WHEN "01" => r_reg <= sin_r & r_reg(3 DOWNTO 1); -- Shift Right
        WHEN "10" => r_reg <= r_reg(2 DOWNTO 0) & sin_l; -- Shift Left
        WHEN "11" => r_reg <= pin;                      -- Parallel Load
        WHEN OTHERS => NULL;
```


© Biraj

```
      END CASE;
    END IF;
  END PROCESS;
  pout <= r_reg;
END behavioral;
```

4. FSM Sequence Detector for Pattern ”1011” (Overlapping Mealy Machine)

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;

ENTITY seq_detector_1011 IS
  PORT (
    clk   : IN  STD_LOGIC;
    rst_n : IN  STD_LOGIC;
    din   : IN  STD_LOGIC;
    dout  : OUT STD_LOGIC
  );
END seq_detector_1011;

ARCHITECTURE fsm_arch OF seq_detector_1011 IS
  TYPE state_type IS (S0, S1, S10, S101);
  SIGNAL state_reg, next_state : state_type;
BEGIN
  -- Process 1: State Register (Synchronous)
  PROCESS (clk, rst_n)
  BEGIN
    IF (rst_n = '0') THEN
      state_reg <= S0;
    ELSIF rising_edge(clk) THEN
      state_reg <= next_state;
    END IF;
  END PROCESS;

  -- Process 2: Next State and Output Logic (Combinational)
  PROCESS (state_reg, din)
  BEGIN
    dout <= '0';
    next_state <= state_reg;
    CASE state_reg IS
      WHEN S0 =>
       IF (din = '1') THEN next_state <= S1; ELSE next_state <= S0; END IF;
      WHEN S1 =>
       IF (din = '0') THEN next_state <= S10; ELSE next_state <= S1; END IF;
      WHEN S10 =>
      IF (din = '1') THEN next_state <= S101; ELSE next_state <= S0; END IF;
      WHEN S101 =>
        IF (din = '1') THEN
          dout <= '1'; -- Pattern "1011" detected!
```


© Biraj

```
          next_state <= S1; -- Overlapping: prefix "1" retained
        ELSE
          next_state <= S10;
        END IF;
    END CASE;
  END PROCESS;
END fsm_arch;
```


© Biraj

7. Quick Revision Page

Key VHDL Constructs and Static Timing Equations

- **Setup Time Constraint:**

$$T_{\text{clk}} \geq t_{cq} + t_{\text{comb(max)}} + t_{\text{setup}} - t_{\text{skew}} \implies f_{\text{max}} = \frac{1}{T_{\text{clk(min)}}}$$

- **Hold Time Constraint:**

$$t_{cq} + t_{\text{comb(min)}} \geq t_{\text{hold}} + t_{\text{skew}} \quad (\text{independent of } T_{\text{clk}})$$

- **VHDL Entity-Architecture Skeleton:**

```
ENTITY e IS PORT (...); END e;  ARCHITECTURE a OF e IS ... BEGIN ... END a;
```

One-Line High-Yield Definitions

- **Logic Synthesis:** The automated translation of abstract RTL code into an optimized gate netlist mapped to standard library cells.
- **Static Timing Analysis (STA):** A vector-independent CAD technique verifying setup and hold timing across all paths.
- **Clock Skew:** The difference in clock arrival times at two distinct registers caused by routing differences.
- **VHDL Entity:** The primary design unit defining external hardware interface ports and generic parameters.
- **VHDL Architecture:** The design unit describing internal hardware dataflow, algorithmic behavior, or structural netlist.
- **SIGNAL in VHDL:** A hardware object representing physical interconnecting wires, scheduled with delta delay.
- **VARIABLE in VHDL:** A local computational object evaluated sequentially and updated instantaneously inside a process.
- **Inferred Latch:** An unintended transparent latch generated by synthesis due to incomplete conditional assignments.
- **Sensitivity List:** The list of trigger signals in a process header; the process evaluates when any listed signal changes.
- **DRC (Design Rule Check):** CAD verification ensuring physical layout geometries comply with foundry manufacturing rules.

© Biraj

Frequently Asked / Viva Questions

1. **What is the difference between a SIGNAL and a VARIABLE in VHDL?**

   A `SIGNAL` models physical electrical wires, is declared in the architecture declarative section, and updates with a scheduling delay (<=). A `VARIABLE` models local scratchpad data, is declared strictly inside a process/subprogram, and updates instantaneously (:=).

2. **What causes an unintended transparent latch to be inferred during synthesis?**

   Incomplete `IF` statements (missing `ELSE`) or incomplete `CASE` statements (missing `WHEN OTHERS`) in combinational processes force synthesis tools to generate latches to preserve previous values.

3. **Why is Static Timing Analysis (STA) preferred over dynamic timing simulation?**

   STA exhaustively checks setup and hold constraints across every single timing path in the chip without requiring test stimulus vectors, achieving 100% path coverage exponentially faster.

4. **What is the physical meaning of the 9 values in IEEE STD_LOGIC?**

   It provides realistic simulation of digital states: `'U'` (uninitialized), `'X'` (forcing unknown), `'0'` (strong 0), `'1'` (strong 1), `'Z'` (high impedance), `'W'` (weak unknown), `'L'` (weak 0 / pull-down), `'H'` (weak 1 / pull-up), and `'-'` (don't care).

5. **What is the difference between synchronous and asynchronous resets in VHDL?**

   An asynchronous reset appears in the process sensitivity list and executes immediately regardless of clock. A synchronous reset does not appear in the sensitivity list and evaluates strictly on the clock edge.

6. **What is the Gajski-Kuhn Y-Chart?**

   A geometric framework visualizing VLSI design across three representations (Behavioral, Structural, Physical) intersecting through multiple levels of abstraction.

7. **What is the purpose of Layout Versus Schematic (LVS) verification?**

   LVS extracts device connectivity directly from fabricated geometric mask layouts and verifies that it electrically matches the synthesized circuit schematic 1-to-1.

8. **Why cannot signals assigned inside a process be read immediately within the same cycle?**

   Because signal assignments inside a process are non-blocking and scheduled for update only after the process suspends (after delta delay δ).

9. **What is the difference between functional simulation and timing simulation?**

   Functional simulation verifies RTL logic algorithms with ideal zero delay. Timing simulation annotates post-layout parasitic delays (via an SDF file) to verify real hardware delays.

10. **What is the role of Clock Tree Synthesis (CTS) in physical design?**

    CTS creates a balanced distribution network (such as an H-tree) to ensure clock edges arrive at all flip-flops simultaneously, minimizing clock skew.

© Biraj

Frequently Asked / Viva Questions (contd.)

11. **Explain the difference between positional association and named association in port mapping.**

    Positional association connects signals based on strict order in the entity port list. Named association links signals explicitly by name (`port_name => signal_name`), preventing wiring errors.

12. **What is the difference between synthesizable and non-synthesizable VHDL constructs?**

    Synthesizable constructs map directly onto physical standard cells (e.g., gates, flip-flops). Non-synthesizable constructs (e.g., `wait for 10 ns`, file I/O) are used solely in simulation testbenches.

Summary Matrix: Comparison of VHDL Modeling Styles

Feature	Dataflow Modeling	Behavioral Modeling	Structural Modeling
Primary Construct	Concurrent signal assignments	`PROCESS` with sequential statements	`COMPONENT` and `PORT MAP`
Execution Semantics	Concurrent (Event-driven)	Sequential inside process	Concurrent interconnection
Hardware Analogy	Boolean logic gates and data flow	Algorithmic behavior, state machines	Schematic netlist wiring
Typical Commands	`<=`, `when-else`, `with-select`	`if-then-else`, `case-when`, loops	`COMPONENT`, `PORT MAP`
Best Suited For	Small combinational modules, muxes	Complex controllers, counters, FSMs	Top-level integration, hierarchies

© Biraj